

GLFM: Geometry-Regularized Latent Flow Models for Single-Cell Perturbation Prediction

David Crair
david.crair@yale.edu

Advisor: Smita Krishnaswamy
smita.krishnaswamy@yale.edu

*A Senior Thesis as a partial fulfillment of requirements
for the Bachelor of Science in Computer Science*

Department of Computer Science
Yale University
Apr 30, 2026

Contents

1	Introduction	6
1.1	The Challenge of Drug Discovery	6
1.2	Single-Cell Transcriptomics	6
1.3	The Generalization Problem	6
1.4	Simulation-Based vs Simulation-Free Training	7
1.5	Summary of Contributions	7
2	Background	9
2.1	Single Cell RNA-Seq Data and Processing	9
2.2	Autoencoders	9
2.3	Continuous Dynamics (OT, FM, and ODEs)	10
2.3.1	Optimal Transport (OT)	10
2.3.2	Flow Matching (FM)	10
2.3.3	(Neural) ODEs	11
3	Related Work	12
3.1	Early Perturbation Models	12
3.2	Foundational Models	12
3.3	Flow Based Models	13
3.4	Geometry-Aware Models	13
3.5	Synthesis	14
4	Methodology	15
4.1	Implementation Details	15
4.2	Datasets	15
4.3	Evaluation Protocol	16
4.4	Geometrically Regularized Autoencoder	16
4.4.1	Autoencoder Training Objective	16
4.5	Condition Encoder	17
4.6	Flow Matching (FM) Model	17
4.7	Neural ODE	18
4.7.1	Forward Integration of Neural ODE	18
4.7.2	Distributional Matching via Optimal Transport	18
4.7.3	Auxiliary Losses	19
4.7.4	Combined Neural ODE Objective	19
4.8	Baselines	20
4.9	Metrics	20
5	Results	21
5.1	Sciplex3 Context Generalization	21
5.2	Dimensionality Reduction as a Bottleneck on Model Performance	22

5.3	Experimental Flow Matching and Neural ODE Results in Different PCA Dimensions	22
5.4	Flow Matching Model Performance as a Function of Geometric Regularization Strength of Autoencoder	23
5.5	Latent-Space Reconstruction Limits in Different Latent Spaces	25
6	Discussion	29
6.1	PCA/AE Bottleneck	29
6.2	What Geometric Regularization to Use?	30
6.3	Limitations	31
7	Conclusions	33
8	Future Work	35
9	Appendix	39
A	Theoretical Derivations, Baselines, and Extra Definitions	39
A.1	Setup and Notation	39
A.2	Idealized PCA Scaling for Cosine LogFC	40
A.3	Idealized PCA Scaling for W_2	41
A.4	Idealized PCA Scaling for $\overline{W}_1$	41
B	Single Cell Dataset Preprocessing	42
C	PCA Space for Training	42
D	Experimental Setup	43
E	Graph Based Models	43
F	Metrics	44
F.1	Maximum Mean Discrepancy	44
F.2	(Squared) Wasserstein Distance (per-gene W_1 and multivariate W_2^2)	44
F.3	Cosine LogFC	45
F.4	DEG Top- k Recall Metric	45
G	Baselines	45
G.1	No Effect Baseline	45
G.2	Linear Baseline	45
G.3	Decoder-Only Baseline	46
G.4	Additive Baseline	46
G.5	Perturbation Mean Baseline	46
G.6	Context Mean Baseline	47
H	Autoencoder Losses	47
H.1	Distance Preservation Loss	47
H.2	Negative Binomial Negative Log Likelihood Loss	48
H.3	Vanilla (MSE) Autoencoder Training Objective	48
I	Condition Encoder Details	49
J	Toy Dataset	49
J.1	Toy Dataset Methods	49
J.2	Toy Dataset Results	49
J.3	Toy Dataset Flow Matching Visualization	50
K	Dimensionality Reduction as a Bottleneck on Model Performance Contd.	50

L	Additional Figures	52
M	Additional Tables	52
M.1	Comparison with PerturBench Results	52

GLFM: Geometry-Regularized Latent Flow Models for Single-Cell Perturbation Prediction

David Crair

Abstract

Predicting heterogeneous responses to drug perturbations is valuable in drug discovery by reducing wet lab burden and enabling faster development of new therapeutics through in-silico experimentation. One key challenge is generalizing known perturbation effects to unseen cell populations, like new patients, complex tissues, or new types of cancer, where only a subset of drugs have been tested. In GLFM, we explore how generative flow models (flow matching and neural ODEs) can be used in the latent space of a geometrically-regularized autoencoder to perform single-cell drug perturbation modeling. We condition these models on sample and drug covariates like cell type and drug dose to enable flexible and generalizable modeling via a learned condition encoder. We primarily evaluate on the SciPlex3 dataset, containing 3 human cancer cell lines and around 200 drug treatments in each cell type. We found that the choice of autoencoder reconstruction loss and latent space largely dominates the effect of geometric regularization, although geometric regularization in the vanilla autoencoder latent space yields consistent improvements and stabilizes autoencoder training. We also derive upper bounds on Cosine LogFC, $\bar{W}_1$ and W_2^2 metrics as functions of cumulative variance explained under PCA, showing that MSE reconstruction accuracy is an upper bound on W_2^2 for autoencoder models. Counterintuitively, we find that flow matching models in lower dimensional PCA space (d=16) outperform similar models in higher dimensional PCA space (d=256) despite the higher latent-space reconstruction ceiling on performance in the higher dimensional space. Finally, we recommend metric-informed autoencoder design, such as using MSE loss when W_2^2 is a metric (because it directly upper-bounds W_2^2) and suggest computation of encode-then-decode reconstruction ceiling for perturbation prediction and single-cell trajectory-inference modeling before investing in flow model tuning, to avoid optimization in spaces with capped ceilings on metrics.

1 Introduction

1.1 The Challenge of Drug Discovery

Transcriptomic data is valuable to both biology and drug discovery to see heterogeneous effects of drug and gene perturbations, but current models often fail to generalize known perturbation effects to unseen biological contexts (e.g. cell lines or patients). The ability to generalize to underrepresented contexts reduces wet lab burden and allows for transfer of knowledge, from contexts that are less challenging to produce data for, to contexts that are infeasible or otherwise prohibitively expensive to generate data in.

The ultimate goal in single cell modeling would be to be able to accurately predict transcriptomic response to completely unseen perturbations, like arbitrary gene knock-out, drug perturbation, or biologics. This would unlock incredible value, enabling accurate screening of compounds in drug discovery or interrogating gene programs for low marginal cost.

1.2 Single-Cell Transcriptomics

scRNA-seq is particularly suited to capture the cell-level heterogeneity present in relevant systems (e.g. human tissue) but also presents its own challenges. One challenge of single cell data is that it is an inherently destructive process. This means that we cannot measure a particular cell multiple times in single cell sequencing, such as before and after a drug perturbation, or in different cell cycle stages. This motivates the use of distributional metrics and approaches in modeling effects in single cell data. Another challenge of single cell data is the high dimensionality of sequencing tens of thousands of genes. This results in a high dimensional vector representation for each cell, with measurements of the counts of each gene for the cell. A third challenge is the sparsity present in the counts. There are often many "real" zeros, when a gene is truly not expressed in a given cell. There are also "technical" zeros from the measurement process, where the sequencing process fails to capture those transcripts.

1.3 The Generalization Problem

One core task in single cell data involves generalizing to contexts where we have limited data. This task has emerged in the past few years as one of the most studied tasks in deep learning for single cell modeling. The first public competition regarding this task was a 2023 NeurIPS challenge and recently (end of 2025) the inaugural Virtual Cell Challenge concluded, bringing a prize pool of over \$250,000 [1, 2, 3]. In the underrepresented context generalization task, one context (e.g. donor, cell type) has data for only a subset of perturbations and all other contexts have data for all or almost all of the perturbations. Each context, including the underrepresented one, will always have control data. The task is to predict the response of the held-out perturbations in the underrepresented context,

which is a generalization task. The idea is that the model can learn generally how the cells in the underrepresented context "behave" (i.e. respond to perturbations) in the subset of the perturbation data that is present for that context and infer the response to the unseen perturbations in that context based on how those perturbations affect the contexts where there is data.

1.4 Simulation-Based vs Simulation-Free Training

When modeling single cell data that changes (i.e. generating trajectories or predicting effects), there are two main ways to train a Neural ODE model to predict instantaneous velocity. The simulation-based methods run ODE simulations during training to get simulated endpoint or intermediate marginal predictions, compute loss at those marginals, then backpropagate through the ODE solver or using the adjoint method. To restate, the model is trained by comparing predicted to ground truth distributions at steps where the ground truth distributions are known. This is accurate and more direct, but it is computationally expensive to run the ODE solver during every training step, since many calls to the neural network are required. The main simulation-free method is flow matching, which avoids using an ODE solver during training. Instead of simulating the trajectory and evaluating at marginals to train a model to predict velocity, the flow matching model more directly regresses to match a computed velocity field. The simulation-free method has a big advantage over the simulation-based method in that it requires only one model call during each training step, but distributional-based loss terms, like used in [4] [5] [6] are difficult to translate to simulation-free training. In both cases, during inference, an ODE solver is used to integrate from a source cell to a cell at a later time.

1.5 Summary of Contributions

We propose a novel combination of existing approaches to modeling single-cell transcriptomic data for perturbation prediction. By flattening the latent space of an autoencoder and performing flow matching in this latent space, the straight-line flows that flow matching trains on are intended to approximate geodesics on the gene expression manifold. This may improve model performance by ensuring that interpolated cellular trajectories respect the underlying biological manifold structure. Even though the task is not trajectory inference as in [6] or [7], we theorize that this flattening will still be beneficial for the perturbation prediction task as sampled linear interpolations between source and target distributions as done in flow matching will correspond to logical biological states on the manifold. We refer to this framework as GLFM: Geometry-Regularized Latent Flow Models.

In this paper we make the following key contributions:

- Apply flow matching (FM) and neural ODE models in the latent space of a geometrically regularized autoencoder (AE) for single-cell perturbation modeling, comparing simulation-free and simulation-based generative flow models

- Derive theoretical upper bounds on Cosine LogFC, W_2 and mean per-gene W_1 as functions of cumulative variance explained under PCA reconstructions at different dimensions, showing that MSE reconstruction loss for AEs directly minimizes the upper bound on W_2
- Show empirically that the choice of AE loss dominates the effect of geometric regularization, but does not eliminate the benefit of geometric regularization
- Find that moderate to strong distance preservation regularization in the latent space of a vanilla AE (MSE loss) improves FM and Neural ODE performance in the latent space of the AE and stabilizes AE training
- Identify a counterintuitive relationship between PCA dimensionality and FM performance, showing FM model performance does not consistently improve with increased upper bounds on model performance

2 Background

2.1 Single Cell RNA-Seq Data and Processing

Single Cell RNA-seq data is a type of genomics data which quantifies the active gene expression at a per-cell level. For example, given a sample containing a large number of human cancer cells, quantifying them with single cell RNA-seq will tell you roughly per-cell transcript counts per gene, capturing cell-to-cell heterogeneity, unlike bulk RNA-seq, which averages over all cells in the sample. Single cell RNA-seq works by dissociating the sample into single cells, isolating the individual cells, tagging the RNA in each cell with a cell-specific barcode, and sequencing the RNA. This produces a matrix, for each sample, of counts of genes for each cell.

From the counts matrix, the typical single cell processing pipeline is to first normalize the total number of counts to some fixed number, typically 10k, then apply the $\log_1 p$ transform to all entries ($\log(1 + x)$). For our purposes, we then compute highly variable genes (HVGs) and differentially expressed genes (DEGs) to subset the large number of genes present in the data to a more meaningful and manageable subset. Highly variable genes are typically computed by fitting a mean-variance trend across genes and picking the top genes with variance that exceeds the prediction based on the mean expression. Differentially expressed genes are typically computed by a per-gene statistical test (t-test or Wilcoxon), comparing a group of interest (cells perturbed by a specific drug) to a reference group (control), ranking by score, and taking the top genes.

2.2 Autoencoders

Autoencoders are a type of machine learning model that produce a condensed representation of data through the use of two networks, an encoder and a decoder. The encoder is a neural network that compresses the data from high dimensions to a lower dimension. The space in which these compressed representations live is the latent space of the autoencoder. The decoder is a neural network that decompresses the data from the latent space back to the ambient space of the original data. The encoder and decoder networks of an autoencoder are typically trained jointly, using the mean squared error between the original input and its reconstruction after passing through the encoder and decoder. Equation 2.1 shows the two functions of the autoencoder, E_ϕ , the encoder and D_ψ , the decoder. $x \in \mathbb{R}^d$ is the input, $z \in \mathbb{R}^k$ is the latent representation and $\hat{x}$ is the reconstruction. The overall model of an autoencoder is $\hat{x} = (D_\psi \circ E_\phi)(x)$.

$$\begin{aligned} z &= E_\phi(x) \\ \hat{x} &= D_\psi(z) = D_\psi(E_\phi(x)) \end{aligned} \tag{2.1}$$

2.3 Continuous Dynamics (OT, FM, and ODEs)

2.3.1 Optimal Transport (OT)

Optimal transport is a way to compare two distributions by finding the cheapest way to move mass from one distribution to the other. In the continuous setting, the optimal transport problem is formulated in Equation 2.2, where μ is the source distribution, ν is the target distribution, $c(x, y)$ is the cost of moving mass from x to y and $\Pi(\mu, \nu)$ is the set of all couplings with marginals μ and ν .

$$\min_{\pi \in \Pi(\mu, \nu)} \int_{\mathcal{X} \times \mathcal{Y}} c(x, y) d\pi(x, y) \quad (2.2)$$

A common choice for the cost for optimal transport is Euclidean distance, where $c(x, y) = \|x - y\|^2$. This results in the squared Wasserstein distance as the infimum of costs of couplings of μ and ν :

$$W_2^2(\mu, \nu) = \inf_{\pi \in \Pi(\mu, \nu)} \int \|x - y\|^2 d\pi(x, y) \quad (2.3)$$

In the discrete case, if $a \in \mathbb{R}^n$ and $b \in \mathbb{R}^m$ are probability vectors and $C \in \mathbb{R}^{n \times m}$ is the cost matrix, then we have the OT form:

$$\min_{P \in U(a, b)} \langle P, C \rangle \quad (2.4)$$

where $U(a, b)$ is the set of all feasible transport plans between the discrete distributions a and b , written as:

$$U(a, b) = \{P \in \mathbb{R}_+^{n \times m} : P\mathbf{1}_m = a, P^\top \mathbf{1}_n = b\} \quad (2.5)$$

where P is the transport plan and

$$\langle P, C \rangle = \sum_{i=1}^n \sum_{j=1}^m P_{ij} C_{ij} \quad (2.6)$$

2.3.2 Flow Matching (FM)

Flow matching (FM) is a type of generative model that is able to generate new samples from a target distribution $q \in \mathbb{R}^d$ by training on some dataset of samples [8]. The way FM models are trained is to regress to the velocity field defined as the instantaneous velocities along linear interpolations between samples from a source distribution p_0 and a target distribution p_1 , along the probability path p_t . To generate a new sample after training, we can solve the Ordinary Differential Equation (ODE) based on the velocity field predicted by the flow matching model (neural network). In general, the path between source and target distributions can be anything, but it is most commonly a linear interpolation and often the specific samples from the source and target distribution that we interpolate between are determined using the optimal transport plan between the distributions. We can define this linear path for random variable $X_t \sim p_t$ by taking a linear combination of random

samples $X_0 \sim p$ from our starting distribution and $X_1 \sim q$ from our ending distribution as follows:

$$X_t = tX_1 + (1 - t)X_0 \sim p_t \quad (2.7)$$

The ODE for flow matching is defined based on a time-dependent vector field $u: [0, 1] \times \mathbb{R}^d \rightarrow \mathbb{R}^d$, where the velocity field is modeled by the neural network. This gives rise to a time-dependent flow $\varphi: [0, 1] \times \mathbb{R}^d \rightarrow \mathbb{R}^d$, which we define as

$$\frac{d}{dt}\varphi_t(x) = u_t(\varphi_t(x)) \quad (2.8)$$

where $\varphi_t(x) := \varphi(t, x)$ and $\varphi_0(x) = x$. Now, our loss for flow matching is:

$$\mathcal{L}_{\text{FM}}(\theta) = \mathbb{E}_{t, X_t} \|u_t^\theta(X_t) - u_t(X_t)\|^2 \quad (2.9)$$

where $t \sim \mathcal{U}[0, 1]$ and $X_t \sim p_t$.

Note: Most of this FM explanation is summarized from Lipman et al. [8].

2.3.3 (Neural) ODEs

Neural ODEs, introduced by Chen et al. [9] in 2019, parameterize the time derivative of an ordinary differential equation (ODE) with a neural network:

$$\frac{dx_t}{dt} = f_\theta(x_t, t)$$

where the time derivative $\frac{dx_t}{dt}$ at a specific time t and location x_t is defined to be the output of a neural network f , with parameters θ , which has current location x_t and current time t as the inputs to the neural network. When considering f_θ across the entire space, it operates like a time-dependent vector field.

To train Neural ODE models, we use an ODE solver or ODE solving technique (e.g., Euler method) to integrate the ODE forward from the source distribution (control cells) to the target distribution (perturbed cells). Then, we compute a loss based on some combination of the endpoint predicted distribution (like comparing to the ground truth endpoint distribution) and loss terms involving the intermediate velocities or trajectories predicted by the ODE (like penalizing the magnitude of the predicted velocities along the trajectory). In practice, many Neural ODE implementations (including ours) backpropagate directly through the ODE solver because of numerical-accuracy issues on the adjoint method [10, 9]. Neural ODEs require many function (neural network) evaluations per sample when training, compared to Flow Matching, which requires only one function evaluation per sample when training. This can make Neural ODEs much slower to train than flow matching models, but they have no assumption that the interpolation between control and target samples is linear, as is typically used in Flow Matching training.

3 Related Work

3.1 Early Perturbation Models

One category of early models for drug response prediction was primarily based on disentangling the basal state of cells and the effects of perturbations and covariates, to later recombine them in unseen combinations. In their CPA model, Lotfollahi et al. [11] introduce an adversarial network to force the embeddings of these basal states and covariates to be separate. They train a discriminator classifier to predict perturbation and covariates based on the basal cell state and backpropagate on a combination of a reconstruction loss and the discriminator error. While the model is able to make predictions of gene expression values that are highly correlated with the ground truth, it has been shown to struggle predicting higher distributional moments correctly, as seen in poor MMD PCA performance on drug combination prediction and on large datasets with complex covariates, as shown by Wu et al. [12] in PerturBench. Another problem with CPA is that Wu et al. showed that removing the adversarial component from CPA improves model performance over unmodified CPA across most metrics when predicting perturbation effects, particularly in the covariate transfer task on the Srivatsan20 dataset [13]. This shows that the core idea of CPA, disentangling the latent space with an adversarial network, does not improve performance, significantly undermining this type of model.

3.2 Foundational Models

Recently, transformer-based foundational models have been fine-tuned to perform perturbation modeling for single cell data. First, Cui et al. [14] developed scGPT, a relatively standard transformer model, treating each gene as a token, where a cell is analogous to a sentence. To predict perturbations, a condition token is appended to each gene embedding to indicate whether it was perturbed. Then, the transformer model is run and the resulting gene embeddings are decoded to get back predictions of log1p transformed gene expression. Another transformer-based model, CellFM, by Zeng et al. [15], trains a larger model on a larger dataset and updates the transformer architecture to use a modified RetNet framework. Both scGPT and CellFM train the model to recover the gene expression for the masked genes, by updating the masked gene tokens using the context of the other genes. CellFM and scGPT also both contain a special token to represent the entire cell, using a loss that combines predicting gene expression for each gene from this cell token as well as predicting gene expression for a single gene from its embedding. CellFM also differs from scGPT in how it handles perturbation prediction. CellFM is used to perform perturbation prediction by using the GEARS architecture but replacing the gene-embedding graph neural network with the gene embeddings produced by the CellFM model.

3.3 Flow Based Models

Following the development of Flow Matching for Generative Modeling by Lipman et al. [16], flow matching has been adopted for perturbation prediction in single cell modeling. One model, CellFlow [17], neatly encodes covariates and performs a conditioned flow matching in a simple latent space for perturbation prediction. CellFlow performs flow matching in a latent space, either in PCA space or the latent space of a variational autoencoder (VAE). The generated cells are then decoded to gene space, which allows metrics to be computed in that space. The model uses OT within each mini-batch during training, conditioning the flow matching model on a condition embedding. The condition embedding is generated by aggregating covariate encodings and applying an MLP, matching the DeepSet setup [18]. The covariate encodings are either learned or pulled from existing models, like in the case of proteins, where the model uses ESM2 embeddings. Klein et al. [17] show that all methods have comparable performance for low doses in the SciPlex3 dataset: none outperform the baseline. In the SciPlex3 dataset, one of the perturbation covariates is dosage, ranging from 10 nM to 10 μ M.

3.4 Geometry-Aware Models

There is another class of models for single cell data, not specifically for perturbation prediction, that take into account the geometry of the data in the setup of the models. First, Huguet et al. [4] developed MIOFlow, a model that performs trajectory inference in single cell data using a Neural ODE in the latent space of a geometrically regularized autoencoder. The autoencoder is trained to minimize the difference between the euclidean distance, in latent space, and the diffusion geodesic distance, in ambient space, between pairs of points, where the diffusion geodesic distance is the shortest distance between two points on the data manifold. MIOFlow then trains a Neural ODE model in the latent space of the autoencoder to predict trajectories of single cell data. The neural network trained predicts the instantaneous velocity at a given position and the loss functions are computed at the distribution for each timepoint, comparing the ground truth and predicted distributions. To get a predicted distribution of data at a specific timepoint, the model uses an ODE solver to integrate the velocity for each cell in the source distribution to the target distribution. There is an additional diffusion term that can be added to the trajectories to model diffusion, resulting in an SDE. MIOFlow performs well on the task of inferring the distribution of data when leaving one timepoint out from the sequence of timepoints, improving across many metrics and training orders of magnitude faster than previous methods. The three loss terms used to train the Neural ODE, in brief, encourage the paths generated to be smooth, stay on the manifold, and match the known distributions. One key disadvantage of MIOFlow is that it requires ODE simulation during training and inference, which is computationally expensive. This is true of any model which relies on integrating an ODE during training.

GAGA [6] is an extension of MIOFlow which derives a warped metric on the data to enable computation of geodesics, which are used for data generation. The autoencoder is very similar to MIOFlow, using the same idea of matching geodesic distances in the ambient space with euclidean distances in the autoencoder latent space. The GAGA

model exhibits vastly improved performance on trajectory inference for the Cite and Multi datasets, evaluating W_1 distance between the ground truth and predicted distributions for a held-out timepoint. GAGA also enables uniform data generation on a manifold and on-manifold interpolation, but these tasks are less directly relevant to perturbation modeling. Although GAGA performs well on trajectory inference, it has not been evaluated on perturbation prediction.

CFlows, by Sun et al. [19], updates the MIOFlow framework to enable modeling of cell growth and death through an additional network that predicts the growth rate of a cell continuously in time.

FlatVI [7] is a novel approach to enforcing Euclidean geometry in the latent space of an autoencoder. Unlike the aforementioned models, FlatVI uses a variational autoencoder (VAE) instead of a standard autoencoder as in MIOFlow, Cflows, and GAGA. This follows from the work of Lopez et al. [20] in scVI, where the decoder of the VAE maps points in the latent space to a statistical manifold representing multivariate negative binomial distributions. From there, the mean and dispersion parameters of a negative binomial distribution for each gene can be decoded. This additionally allows for generation of a distribution of cells from one point in the statistical manifold or latent space, by sampling from the negative binomial distributions. FlatVI regularizes the pullback metric of the decoder of the VAE to make straight paths in latent space correspond to geodesic paths along the probability manifold of the decoded space. This is done by regularizing the pullback metric in the latent space induced by the VAE decoder to be a scaled isometry everywhere. Unlike GAGA, the FlatVI model performs flow matching, in the latent space of the VAE, to learn cellular trajectories. FlatVI is also not trained to do perturbation modeling, but the authors mention they are planning on doing this extension.

3.5 Synthesis

In the past five years, there has been rapid development of single cell models, looking at both predicting responses to perturbations and modeling cellular trajectories. In the cellular trajectory models, it has become apparent that regularizing the latent space in which the interpolation is performed to be more Euclidean ("flat") is beneficial to model performance. Yet, these same regularization techniques have not been applied to perturbation modeling even though the same core rationale holds. When training models by constructing linear interpolations in the latent space, there is an implicit assumption that the latent space has Euclidean geometry. This produces suboptimal results when this assumption is violated, as the straight line paths through latent space no longer correspond to geodesic paths on the data manifold. This assumption is agnostic of performing trajectory inference or perturbation prediction and can be assumed to be violated for single cell data.

When multiple drug doses are present in the data for a single perturbation, the perturbation modeling task becomes analogous to the cellular trajectory modeling, but the trajectory is over drug dose instead of time, further motivating the use of geometric regularization in perturbation prediction models.

See Appendix E for additional information on graph-based perturbation prediction models.

4 Methodology

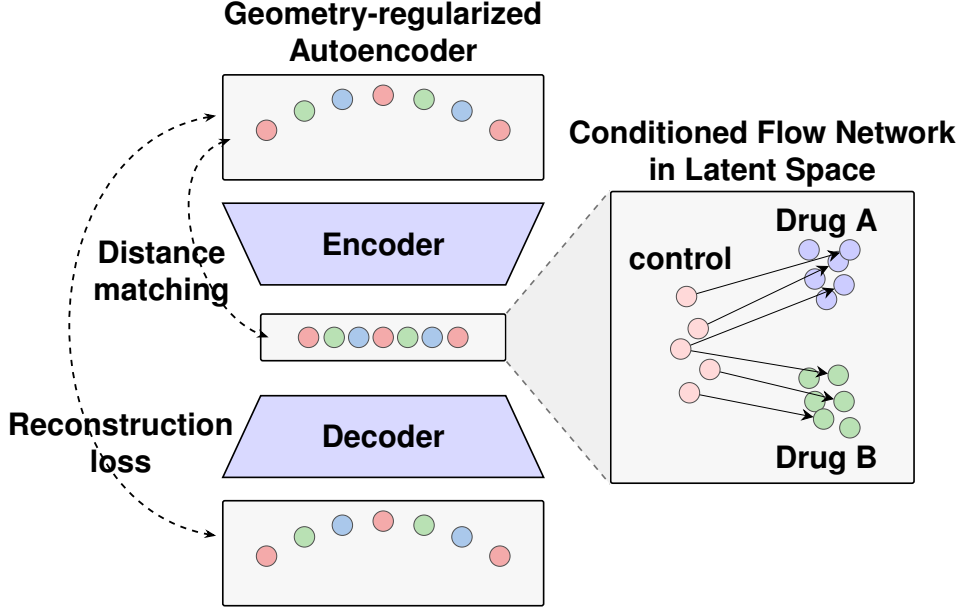


Figure 4.1: Schematic of GLFM. GLFM performs perturbation prediction by learning a time-dependent vector field using a Neural ODE $f_{\theta}(x_t, t, \mathcal{B})$ or Flow Matching model $u_{\theta}(x_t, t, \mathcal{B})$, conditioning on the perturbation and covariates, $\mathcal{B}$. This modeling is performed in the latent space of a geometrically regularized autoencoder.

4.1 Implementation Details

GLFM is implemented with PyTorch Lightning and hyperparameter optimization is performed with Optuna using Hydra for configuration.

4.2 Datasets

We typically train models on a 100k cell subset of SciPlex3, with cells chosen to preserve control/perturbed cell ratio per cell-type, but train autoencoders on a further 50k subset of those 100k cells for performance. See Appendix B for details on how the dataset is preprocessed. We split the dataset by holding out 50% of K562 perturbations, such that all doses are held-out together for a given perturbation. We then do a 90/10 train/val split on the remaining 50% of the data. If we hold out a perturbation for the val or test set, we hold out all doses of that perturbation in the same set. This means that we are never testing generalization to an unseen dose of a perturbation that is seen at other doses in the held-out cell type, we just evaluate on cell-type transfer task in the underrepresented cell type, like described above.

4.3 Evaluation Protocol

For our evaluation, we do not perform cross validation, unless stated otherwise, and just report metrics on the single K562-held-out split, averaged across 3 training seeds. When we train a generative flow model in a different space than the space we are performing evaluations, we will do the inverse transformation to bring the predictions back to the correct space for computing the metrics. For example, if we are computing the metrics in $\log_{1p}$ space, but the Flow Matching model trains in PCA space, we will get the model predictions in the PCA space, then apply the PCA inverse transform to recover predictions in $\log_{1p}$ space. To restate: the generative flow models get as input the data after the dimensionality reduction, perform inference in this lower-dimensional space, then the inverse transformation is applied to the predictions to recover predictions in the original space. This allows for the comparison of models trained in different dimensions of latent space, which matches the procedure from PerturBench [12].

4.4 Geometrically Regularized Autoencoder

Recently, many methods have been developed in generative modeling, particularly with relevance to single cell modeling, with the core idea that the space of prediction for a generative model should contain all the signal possible while being simple, but not limiting, in expressivity. For example, Lu et al. [21] (Pixel Mean Flows) change the task for Flow Matching to predict directly in the ambient space of the data, where the data can be assumed to follow the manifold hypothesis, exhibiting a low-dimensional structure embedded in the high dimensional space, which is a lower-dimensional underlying structure to predict when compared to the original flow matching task, where predicting the instantaneous velocity from noise to data is fully supported in high-dimensions due to the mixing of noise with the underlying data manifold.

Similarly, Palma et al. [7] use a latent manifold regularization to encourage a simple latent geometry of an autoencoder. This allows Flow Matching in the latent space to use simpler linear paths in the latent space that decode to non-linear geodesic paths on the decoded manifold. They also use a negative binomial VAE, which explicitly handles distributional details of single cell data, like overdispersion and discreteness, instead of leaving those properties to have to be learned as part of a vanilla autoencoder.

GLFM combines these ideas: we train an autoencoder (optionally with a negative binomial decoder instead of standard decoder to $\log_{1p}$ space) to explicitly model the discrete counts structure of single cell data, and add geometric regularization to the autoencoder via an additional loss term so that the latent space of the autoencoder is more faithful to the geometry of the underlying data manifold.

4.4.1 Autoencoder Training Objective

The training objective of the geometrically regularized autoencoder is to minimize the two loss terms, a reconstruction term and the mean squared error comparing pairwise Euclidean distances between cells in latent space and the corresponding PHATE potential distances between the same pairs of cells. We also ignore inter-cell-type potential distances to focus on perturbation-induced geometric structure within a cell-type (see Figure

5 to view the PHATE embedding of the SciPlex3 dataset). One important caveat to note is that the PHATE potential distances are computed after performing PCA on the dataset, not in the full ambient gene expression space. Therefore, the distance preservation loss should be interpreted as preserving the geometry of this processed PHATE/PCA representation, rather than the exact geodesics on the full gene expression manifold. This means that the PHATE/PCA representation inherits some of the PCA bottleneck discussed later, however the large number of principal components used in the PCA representation likely captures most of the general data geometry. See Appendix H.1 for the full loss definition and PHATE preprocessing details.

For the reconstruction term, we either use standard mean squared error (MSE) reconstruction, comparing the ground truth log1p normalized gene expression vectors to the decoded gene expression vectors averaged over cells or the negative log likelihood of the data under the parameters of the decoded negative binomial distribution.

4.5 Condition Encoder

We follow Klein et al. [17] and create an encoder for covariates to condition GLFM, enabling conditioned generation of perturbed cells. Given the perturbation, perturbation covariates, and sample covariates, the condition encoder uses learned embeddings to embed each covariate. Then it passes each embedding through a linear layer and concatenates the results. This concatenation is passed through an MLP to produce the final condition embedding. See Appendix I for more details.

4.6 Flow Matching (FM) Model

For applying Flow Matching to the perturbation prediction task, we follow Klein et al. [17] and Palma et al. [7] in using a FM model to flow from control cells to generate perturbed cells in some (latent) space.

We set our starting distribution X_0 to be random samples from our population of control cells and our target distribution X_1 to be random samples from the distribution of perturbed cells, for a single perturbation p , across all sample covariates (e.g. cell types). This is different than how FM models are commonly used for tasks like image generation, where the source distribution is Gaussian noise and the target distribution is the data distribution.

We then couple the control and perturbed cells using the Sinkhorn algorithm for OT. Then, we use a linear interpolation between coupled cells in the control and perturbed distributions, computing the target instantaneous velocity:

$$u_t(X_t) = X_1 - X_0$$

Note that this is not time-dependent, the instantaneous velocity is constant along the linear interpolation from control cell to perturbed cell. We then regress a neural network to match $u_t(X_t)$, parameterized by θ and with inputs X_t and set of cell-specific covariates $\mathcal{B}$. Our flow matching loss is:

$$\mathcal{L}_{\text{FM}}(\theta) = \mathbb{E}_{t, X_t} \|u_t^\theta(X_t, \mathcal{B}) - u_t(X_t)\|^2$$

We optionally sample more control cells than perturbed cells to give more cells for the OT coupling to choose from. If we had exactly B control cells and B perturbed cells in a mini-batch we would force each control cell to be matched with a perturbed cell, which could lead to suboptimal pairings. The multiplier used (i.e., the ratio between control and perturbed cells) is a hyperparameter.

For inference, we use the RK4 ODE solver with the trained neural network u_t^θ to integrate from the control cells X_0 to time $t = 1$ to produce predicted perturbed cells X_1 . This produces a distribution of predicted perturbed cells, one for each control cell in the distribution X_0 .

For the time embedding of the FM model, we use 64-dimensional Gaussian Fourier features, which are non-learnable, following Song et al. [22]. Our exact flow matching neural network consists of an MLP with four hidden layers, each of dimension 256, with ELU activation. We concatenate X_t with the condition embedding and time embedding as the input to the MLP.

4.7 Neural ODE

Unlike flow matching which learns the velocity field u_t^θ by regressing to instantaneous velocity along linear interpolations, Neural ODE learns the velocity field by integrating the ODE using the velocity parameterized by the neural network and comparing the predicted terminal (perturbed) distribution of cells to the observed distribution of perturbed cells.

4.7.1 Forward Integration of Neural ODE

Given a batch of control cells $X_0 = \{x_0^{(i)}\}_{i=1}^B$ and a condition batch $\mathcal{B}$ specifying the perturbation, perturbation covariates, and sample covariates, the Neural ODE integrates each control cell forward from time $t = 0$ to time $t = 1$ using the learned velocity field f_θ :

$$\hat{x}_1^{(i)} = x_0^{(i)} + \int_0^1 f_\theta \left(x_t^{(i)}, t, C_\eta(\mathcal{B}) \right) dt$$

where $C_\eta(\mathcal{B})$ is the learned condition embedding of the condition batch $\mathcal{B}$ from the condition encoder C_η . In practice, we approximate this integral using a numerical ODE solver (e.g., midpoint method, RK4) over discrete time steps $0 = t_0 < t_1 < \dots < t_K = 1$, which yields a trajectory $X = (x_{t_0}, x_{t_1}, \dots, x_{t_K})$ for each cell.

Unlike flow matching, which is simulation-free in not requiring ODE integration during training and only during inference, the Neural ODE method must solve the full initial value problem during every training step. To compute gradients with respect to the network parameters θ during training, we use the backpropagation through a fixed 10-step midpoint integrator.

4.7.2 Distributional Matching via Optimal Transport

The optimal transport loss for the Neural ODE training compares the empirical distribution of predicted perturbed cells $\hat{\rho}_1 = \frac{1}{B} \sum_{i=1}^B \delta_{\hat{x}_1}^{(i)}$ at $t = 1$ to the empirical distribution of

observed perturbed cells $\rho_1 = \frac{1}{B} \sum_{j=1}^B \delta_{x_1^{(j)}}$.

We measure the difference between these two distributions using Sinkhorn divergence, a divergence that is almost W_2^2 but fast and debiased. Given the cost matrix $C_{ij} = \|\hat{x}_1^{(i)} - x_1^{(j)}\|_2^2$, squared Euclidean ground cost, between the predicted and target cells, the OT loss is:

$$\mathcal{L}_{\text{OT}} = \text{OT}_\varepsilon(\rho, \nu) - \frac{1}{2}\text{OT}_\varepsilon(\rho, \rho) - \frac{1}{2}\text{OT}_\varepsilon(\nu, \nu)$$

where

$$\text{OT}_\varepsilon(\alpha, \beta) = \inf_{\pi \in \Pi(\alpha, \beta)} \langle \pi, C \rangle + \varepsilon \text{KL}(\pi \parallel \alpha \otimes \beta)$$

where $\varepsilon > 0$ is the entropic regularization parameter and $\Pi(\alpha, \beta)$ is the set of couplings with marginals α and β .

4.7.3 Auxiliary Losses

Energy Loss Energy loss penalizes the magnitude of the velocity when integrating the Neural ODE from control cells to perturbed cells, enforcing a smooth trajectory.

$$\mathcal{L}_{\text{energy}} \approx \frac{1}{B} \sum_{\hat{x} \in \hat{X}} \int_0^1 \|f_\theta(x_t, t)\|^2 dt$$

In practice, we compute energy loss using a Riemann sum approximation over steps from the ODE solver, following Huguet et al. [4]. For ODE solver discrete time steps from $0 = t_0 < t_1 < \dots < t_K = 1$ as above for a given cell, the exact energy loss used is:

$$\mathcal{L}_{\text{energy}} = \frac{1}{B} \sum_{i=1}^B \sum_{k=1}^{K-1} \left\| \frac{x_{t_{k+1}}^{(i)} - x_{t_k}^{(i)}}{t_{k+1} - t_k} \right\|^2$$

Density Loss Density loss encourages the endpoint to remain near the data manifold, by penalizing distance to the k^{th} nearest neighbor in the dataset, and applies a hinge penalty. This follows the setup in Sun et al. [19].

$$\mathcal{L}_{\text{density}} = \frac{1}{Bk} \sum_{i=1}^B \sum_{j=1}^k \max\left(0, d^{(j)}(x_1^{(i)}) - h\right)$$

where $d^{(j)}(x)$ is the distance from x to its j -th nearest neighbor in the dataset and $h > 0$ is a hinge threshold, a hyperparameter. This loss is zero when the trajectory stays within distance h of the observed data, which acts as a soft manifold constraint. The trajectory inference performed does not use marginals (like drug dose), so the density loss is only computed at the endpoint and not at any marginal, like in [4, 19].

4.7.4 Combined Neural ODE Objective

The full Neural ODE training objective combines the three loss terms, each with a scalar as a hyperparameter:

$$\mathcal{L}_{\text{ODE}} = \lambda_{\text{OT}} \mathcal{L}_{\text{OT}} + \lambda_{\text{density}} \mathcal{L}_{\text{density}} + \lambda_{\text{energy}} \mathcal{L}_{\text{energy}}$$

4.8 Baselines

We implement the following baselines, mostly from Wu et al. [12], and give more detail on each baseline in Appendix G:

- **No Effect Baseline:** predicts perturbations do nothing, returns control cells as predicted perturbed cells
- **Linear Baseline:** learns the effect of each perturbation by adding the output from a linear layer with the condition encoding as input, added to the control cell distribution
- **Decoder-Only Baseline:** MLP that only takes condition batch as input and directly predicts perturbed cells, so cannot learn heterogeneous responses to perturbations
- **Additive Baseline:** Predicts each perturbation effect as the mean perturbation effect across all perturbations and cell lines, and adds that to the control cell distribution
- **Perturbation Mean Baseline:** Predicts the perturbation effect as the mean effect of that specific perturbation in the training data, adding to the control cell distribution
- **Context Mean Baseline:** Predicts that perturbations have no effect, simply returns the average in the training data of the cell type we are predicting the perturbation effect for

4.9 Metrics

We mainly use the following metrics, again mostly from Wu et al. [12] and give more detail on each metric in Appendix F.

- **Wasserstein Distance ($\bar{W}_1$ and W_2^2):** $\bar{W}_1$ measures the mean W_1 distance between the ground truth per-gene distribution and predicted per-gene distribution for a given perturbation, where W_1 measures the work required to move density from one distribution to another (predicted to ground truth). W_2^2 is computed to compare a ground truth distribution of perturbed cells to a predicted distribution of perturbed cells, comparing the full multivariate distribution of cells, which is sensitive to correlations across genes.
- **Cosine LogFC:** Cosine LogFC measures the cosine similarity between the LogFC vector, comparing the mean gene expression for ground truth perturbed cells or predicted perturbed cells to the control cell mean gene expression.
- **Maximum Mean Discrepancy (MMD):** We compute MMD in 50 dimensional PCA space and use a non-linear (RBF) kernel, which measures the discrepancy in a way that is sensitive to higher-order distributional differences.

5 Results

5.1 Sciplex3 Context Generalization

Table 5.1: Benchmark results on held-out K562 perturbations, Sciplex3 dataset. Mean $\pm$ std across perturbations. **Bold** = best, underline = second best per metric. ODE and FM models are trained in the space indicated in parenthesis (autoencoder latent space, or PCA space).

	Model	Mean Gene $\bar{W}_1$ ($\downarrow$)	W_2^2 ($\downarrow$)	Cosine LogFC ($\uparrow$)	MMD ($\downarrow$)
<i>Baselines</i>	No Effect	0.11 $\pm$ 0.030	4085.66 $\pm$ 99.259	-0.00 $\pm$ 0.033	0.07 $\pm$ 0.070
	Additive	<u>0.13 $\pm$ 0.030</u>	4087.57 $\pm$ 96.395	0.02 $\pm$ 0.035	<u>0.08 $\pm$ 0.061</u>
	Perturbation Mean	<u>0.16 $\pm$ 0.041</u>	4131.66 $\pm$ 109.558	0.03 $\pm$ 0.071	<u>0.09 $\pm$ 0.078</u>
	Linear	0.19 $\pm$ 0.032	4296.89 $\pm$ 105.439	0.01 $\pm$ 0.079	0.33 $\pm$ 0.050
	Decoder	0.28 $\pm$ 0.005	2172.76 $\pm$ 66.391	<u>0.18 $\pm$ 0.041</u>	0.25 $\pm$ 0.067
	Context Mean	0.29 $\pm$ 0.005	2210.84 $\pm$ 66.565	<u>0.10 $\pm$ 0.061</u>	0.33 $\pm$ 0.044
	Latent Additive	0.33 $\pm$ 0.007	2570.27 $\pm$ 83.970	0.07 $\pm$ 0.075	0.58 $\pm$ 0.029
<i>ODE</i>	ODE (MSE AE $d=512$)	0.23 $\pm$ 0.010	2216.97 $\pm$ 107.216	0.11 $\pm$ 0.130	0.10 $\pm$ 0.073
	ODE (PCA $d=16$)	0.24 $\pm$ 0.007	2131.55 $\pm$ 76.414	0.20 $\pm$ 0.109	0.11 $\pm$ 0.067
<i>Flow Matching</i>	FM (MSE AE w/ geom reg $w=1e-4$)	0.23 $\pm$ 0.012	2225.03 $\pm$ 82.475	0.14 $\pm$ 0.126	0.08 $\pm$ 0.072
	FM (MSE AE $d=512$)	0.24 $\pm$ 0.013	2185.22 $\pm$ 88.933	0.17 $\pm$ 0.146	0.08 $\pm$ 0.061
	FM (MSE AE w/ geom reg $w=10$)	0.25 $\pm$ 0.009	<u>2144.20 $\pm$ 80.138</u>	0.15 $\pm$ 0.120	0.10 $\pm$ 0.073
	FM (PCA $d=16$)	0.25 $\pm$ 0.008	2148.38 $\pm$ 79.750	0.15 $\pm$ 0.128	0.08 $\pm$ 0.065

In Figure 5.1 and Table 5.1 we show context generalization results on the Sciplex3 dataset across all baselines and a variety of generative flow models. Surprisingly, the approach with the best $\bar{W}_1$, the mean $\bar{W}_1$ distance per-gene between predicted perturbed cells and ground truth perturbed cells is the No Effect baseline, which predicts that perturbations have no effect by returning the control cell distribution. What this means is that the average perturbation effect on any given gene is very small. We find that around 97.7% of counts are zero in the Sciplex3 dataset which decreases to around 88% in the subset of genes we train on, meaning that models can perform very poorly on $\bar{W}_1$ if they predict consistent shifts in gene expression. All generative flow models perform significantly worse than the Additive, Linear, No Effect and Perturbation Mean baselines on $\bar{W}_1$.

For the W_2^2 metric, we find that generative flow model performance is competitive relative to the baselines. The No Effect baseline suffers on this metric, while the Context Mean and Decoder baselines perform pretty well. On W_2^2 , an ODE model trained in 16-d PCA space performs best, although many other models, like FM in 16-d PCA space and FM in geometrically regularized MSE AE latent space perform similarly.

For Cosine LogFC, all the generative flow models outperform all baselines except the decoder baseline. This metric measures if the directionality of the predicted perturbation effect is accurate, which shows that generative flow models are able to learn perturbation effects. Baselines like the No Effect or Perturbation Mean baseline that can't predict perturbation-specific effects and only learn global means are near 0 on this metric. We found the best model to be an ODE trained in 16-d PCA space. Again, many other gen-

erative flow models are competitive on this metric, like all the flow matching models, particularly FM trained in the 512-d latent space of an MSE autoencoder.

On Maximum Mean Discrepancy (MMD; computed in 50-d PCA space), like $\bar{W}_1$, the No Effect baseline is strongest with good performance by the Additive baseline as well. For MMD, however, all of the generative flow models have strong performance, particularly most Flow Matching models.

These results align with recent papers by Csentes, Szalay, and Szalai [23] and Wenter et al. [24] which show that SOTA models still struggle against simple baselines on some single cell modeling tasks. We also evaluate GLFM on the exact PerturBench setup for the Sciplex3 dataset and show results in Appendix M.1.

5.2 Dimensionality Reduction as a Bottleneck on Model Performance

We create a latent-space reconstruction limit on perturbation prediction model performance in any space to explore how dimensionality reduction bottlenecks performance of flow-based generative models. The best a perturbation prediction model in the underrepresented context generalization task can reasonably do with perfect information is predicting the ground truth perturbed cell as the predicted perturbed cell. When this operates in the "base space", the same space in which we derive metrics, this latent-space reconstruction limit will be the actual best possible result for each metric. For example, for the Cosine LogFC metric (defined in Section F.3), when our prediction equals the ground truth perturbed population, $\bar{x}' = \bar{x}_{\text{pert}}$, we have $\Delta\hat{g} = \Delta g$, so $\text{CosineLogFC} = 1$.

Instead, if we perform our "prediction" in some latent space or lower dimensional representation of our data, like the latent space of an autoencoder or PCA space, and treat our predicted cell as passing the ground truth perturbed cell through the dimensionality reduction and then expansion (e.g. encoder then decoder for autoencoder), the latent-space reconstruction limit of this ideal model performance on each metric is constrained by the accuracy of the data reconstruction of the dimensionality reduction we are doing. In practice, we find that the choice of latent dimension has a significant impact on the latent-space reconstruction limit of generative flow models in latent space across all metrics. In Appendix Figure 3 we show that the latent-space reconstruction limit of the cosine LogFC and W_2 metrics are significantly influenced by the dimensions used for PCA, when prediction is performed in PCA space. Cosine LogFC and W_2 each show an approximate square-root relationship with variance explained (x) with cosine LogFC as a power of x and W_2 as a power of $(1 - x)$.

5.3 Experimental Flow Matching and Neural ODE Results in Different PCA Dimensions

In Figure 5.2, we find that when we train the same generative flow model in 16 dimensional PCA space (keeping top 16 PCs) and in 256 dimensional PCA space, the performance is better on the Cosine LogFC and most other metrics in the 16-d space compared to the 256-d space (ODE: 0.167 vs 0.077, FM: 0.159 vs 0.109), despite the latent-space

reconstruction ceiling on the Cosine LogFC metric being around 1.5x higher in 256-d space (0.50 vs 0.34). This finding is highly counterintuitive, even though the latent-space reconstruction ceiling of the Cosine LogFC metric is much higher in the higher dimensional PCA space, training in this space is worse for both generative flow (FM or ODE) models, when training and evaluating on real single cell data (Sciplex3 dataset).

We also find that $\bar{W}_1$ metric performance improves as the dimensionality of the PCA space the generative flow model is trained in increases, which matches our expectation. Specifically on this metric, both models achieve near optimal performance. The flow matching model only falls significantly behind the latent-space reconstruction limit as the PCA dimensions pass 512 and decreases almost monotonically. The performance curve of the Neural ODE model on this metric is less monotonic, increasing mostly monotonically (getting worse) until $d=256$, then improving significantly up to $d=1,024$.

We show in A that the upper bound of $\bar{W}_1$ scales as $\sqrt{1 - \text{CVE}_k}$, the same scaling as W_2 . This indicates that there is something fundamental to this configuration of PCA and generative flow model which results in an improvement in $\bar{W}_1$ performance as the latent space dimensionality increases while the opposite is true for the W_2 metric (performance is worse as latent-space reconstruction limit improves). These two metrics share the same theoretical upper bound, scaling with $\sqrt{1 - \text{CVE}_k}$, but $\bar{W}_1$ matches the upper bound while W_2^2 significantly diverges. One thing to note is that the $\bar{W}_1$ metric only looks at per-gene marginals and not correlation between genes, while W_2 measures the full joint distribution, where correlations matter. We see in 5.1 that the no-effect baseline performs very well on $\bar{W}_1$ and MMD, which implies that this dataset has perturbations that, on average, do not change gene expression significantly.

5.4 Flow Matching Model Performance as a Function of Geometric Regularization Strength of Autoencoder

In Table 5.2 and Figure 5.3 we show Flow Matching (FM) and Neural ODE model performance in the 128-d latent space of a vanilla autoencoder (MSE loss) with different weights of distance-preservation loss, acting as geometric regularization on the latent space. We train on different seeds to get different sets of held-out perturbations, always using the K562 cell-line as the underrepresented context (random 50% of perturbations held out for each seed). We train the FM and ODE models in the same latent space, of the geometrically regularized autoencoder, at different distance loss weights and seeds.

One preliminary detail to note is that a subset of training runs (seeds) for the $w = 0$ autoencoder did not converge well, predicting a per-gene log1p expression value near the mean of the training dataset for every gene. This resulted in poor FM performance around 1/3 of the time on the $w = 0$ distance loss, seen in the large variability at this distance loss weight. We see $\bar{W}_1$ clustered around 0.32 for all generative flow models trained in these "stuck" autoencoders, W_2^2 around 2450, and Cosine LogFC of around 0.05. In Table 5.2, we break out FM and ODE model performance in the latent space of these stuck autoencoders into their own rows to prevent these outliers from bringing down the average for the successfully trained autoencoder runs. When referencing model performance relative to the $w = 0$ baseline subsequently, we usually refer to the set of generative flow models trained in the successfully trained $w = 0$ autoencoders. It is of note that no autoencoders

Table 5.2: Flow matching and neural ODE performance in MSE autoencoder latent vs distance regularization strength. Mean $\pm$ std across seeds shown. **Bold** = best, underline = second best per metric computed across both sections. Stuck rows are autoencoders that collapsed to a constant output identified by AE reconstruction val loss above 0.30 (healthy AE baseline ≈ 0.28 collapsed AE baseline ≈ 0.32). FM and ODE for the same (weight seed) share the same AE latent space.

	distance loss weight	cosine logFC ($\uparrow$)	mean gene $\bar{W}_1$ ($\downarrow$)	W_2^2 ($\downarrow$)	n
<i>Flow Matching</i>	w=0 (overall)	0.1231 $\pm$ 0.0558	0.2634 $\pm$ 0.0431	2251.7 $\pm$ 154.2	9
	w=0 (stuck)	0.0501 $\pm$ 0.0039	0.3207 $\pm$ 0.0003	2457.0 $\pm$ 4.8	3
	w=0 (successfully trained)	0.1596 $\pm$ 0.0138	0.2348 $\pm$ 0.0046	2149.1 $\pm$ 11.5	6
	w=0.00001	0.1559 $\pm$ 0.0094	0.2337 $\pm$ 0.0098	2162.6 $\pm$ 36.1	3
	w=0.0001	0.1562 $\pm$ 0.0148	0.2244 $\pm$ 0.0028	2194.1 $\pm$ 12.1	3
	w=0.001	0.1480 $\pm$ 0.0107	<u>0.2287 $\pm$ 0.0035</u>	2192.5 $\pm$ 2.0	3
	w=0.01	0.1600 $\pm$ 0.0199	0.2356 $\pm$ 0.0017	2169.3 $\pm$ 8.2	3
	w=0.1	0.1528 $\pm$ 0.0114	0.2417 $\pm$ 0.0029	2147.9 $\pm$ 5.3	3
	w=1	0.1609 $\pm$ 0.0206	0.2476 $\pm$ 0.0031	<u>2129.4 $\pm$ 4.4</u>	3
	w=10	0.1772 $\pm$ 0.0201	0.2506 $\pm$ 0.0042	2126.3 $\pm$ 4.5	3
	w=100	<u>0.1689 $\pm$ 0.0201</u>	0.2507 $\pm$ 0.0034	2129.5 $\pm$ 2.2	3
<i>Neural ODE</i>	w=0 (overall)	0.0633 $\pm$ 0.0152	0.2693 $\pm$ 0.0395	2413.2 $\pm$ 136.0	9
	w=0 (stuck)	0.0502 $\pm$ 0.0039	0.3206 $\pm$ 0.0002	2457.0 $\pm$ 4.8	3
	w=0 (successfully trained)	0.0699 $\pm$ 0.0144	0.2437 $\pm$ 0.0115	2391.2 $\pm$ 166.8	6
	w=0.0001	0.0742 $\pm$ 0.0207	0.2332 $\pm$ 0.0022	2211.3 $\pm$ 1.8	3
	w=0.01	0.1019 $\pm$ 0.0174	0.2346 $\pm$ 0.0006	2178.7 $\pm$ 7.4	3
	w=1	0.1312 $\pm$ 0.0064	0.2445 $\pm$ 0.0089	2135.1 $\pm$ 12.3	3
	w=10	0.1344 $\pm$ 0.0179	0.2422 $\pm$ 0.0041	2130.4 $\pm$ 5.5	3
	w=100	0.1332 $\pm$ 0.0149	0.2427 $\pm$ 0.0068	2134.9 $\pm$ 5.1	3

trained with geometric regularization suffered from getting ”stuck” in predicting overly averaged expression, suggesting that this geometric regularization stabilizes autoencoder training to some extent.

For each metric, there is a non-zero distance loss weight that improves FM and Neural ODE performance over the baseline ($w = 0$ MSE autoencoder), but the optimal weight depends on the metric. For $\bar{W}_1$, the optimal distance loss weight is around 10^{-4} , while for W_2^2 and Cosine LogFC improve up through $w = 10$. On the $\bar{W}_1$ metric, we find that for both FM and Neural ODE models, adding geometric regularization improves performance (decreasing $\bar{W}_1$) until 10^{-4} , after which more geometric regularization degrades performance. We see that the FM model trained in latent space of AE with distance loss weight 10^{-4} has the best mean $\bar{W}_1$ across seeds out of all models trained in 128-d MSE AE latent spaces. On W_2^2 , we see optimal FM and ODE model performance at distance loss of $w = 10$. The ODE mean across seeds roughly monotonically decreases until $w = 10$ while FM model mean across seeds dips at 10^{-5} then monotonically increases from 10^{-4} to 10. We see an increase in W_2^2 (worse performance) at weights over 10 for both models. Finally, on Cosine LogFC, the optimal weight is also $w = 10$, but ODE mean Cosine LogFC over seeds increases from $w = 0$ to $w = 10$, while for the FM model, the trend is much more rough. Although these results are not globally optimal on any of the metrics (see Section 5.1), for Autoencoders with a latent dimension of 128,

some level of geometric regularization consistently improves performance in this sweep for both FM and ODE models trained in the latent space.

As shown in 5.2, autoencoders trained with MSE loss directly optimize for the limit of W_2^2 on the training data, so it is a strong result to show that adding geometric regularization via the distance loss term (H.1) improves performance on W_2^2 relative to the baseline without geometric regularization (vanilla MSE, $w = 0$, both models with $d = 128$ latent space).

5.5 Latent-Space Reconstruction Limits in Different Latent Spaces

In Figure 5.4, we train a linear autoencoder and find that the results closely match PCA, achieving similar variance explained as PCA at each dimension, a known result. We find that nonlinear autoencoders plateau at very low variance explained (around 0.25 for MSE autoencoder -0.1 for PHATE autoencoder and -0.06 for NB autoencoder), despite having strictly more capacity than the linear autoencoder of equivalent latent dimension. This plateau is observed across the architectures tested, we find it persists across multiple different nonlinear autoencoder architectures, loss functions, and latent dimensions. We tested training MSE autoencoders (vanilla) with wider hidden dimensions ($\max(512, 2 \times \text{latent_dim})$) to remove capacity in the hidden layers as a bottleneck and did not see improved results on variance explained. The implication for downstream modeling is that the reconstruction ceiling sets an upper bound on any generative flow model operating in these latent spaces. Even a model that perfectly predicted the held-out perturbed gene expression in the 512-d MSE autoencoder latent space would lose 40% of the perturbation signal (measured by Cosine LogFC) compared to the same model in the linear/PCA latent space of the same dimension (512).

Note that the negative variance explained for models using negative binomial reconstruction is an artifact of model training vs testing space. NB autoencoder models (NB AE and PHATE NB AE) learn the parameters of a negative binomial distribution for each gene, which gets sampled to produce count single cell data. We typically evaluate models in the subsetted and log1p transformed space (what we do for this figure for variance explained), which causes negative variance explained for these models. In general, models with negative variance explained are worse at reconstructing the data in the MSE-sense than predicting the global mean. When evaluating the variance explained of the NB autoencoder in counts space, instead of log1p space, we get a variance explained of 0.88 for a model with latent dimension of 16, compared to a variance explained of -0.06 in log1p space for the same model.

For all models other than PCA producing reconstruction $\hat{x}$ for ground truth cells x , we compute variance explained as a scalar R^2 over all cells and genes as follows:

$$\text{VE}(\hat{X}, X) = 1 - \frac{1}{\text{Var}(X) \cdot NG} \cdot \sum_{i=1}^N \sum_{g=1}^G (\hat{x}_i^g - x_i^g)^2$$

where N is the number of held-out cells, G is the number of genes and $\text{Var}(X)$ is the total scalar variance of the ground truth matrix X .

Benchmark performance on held-out K562 perturbations

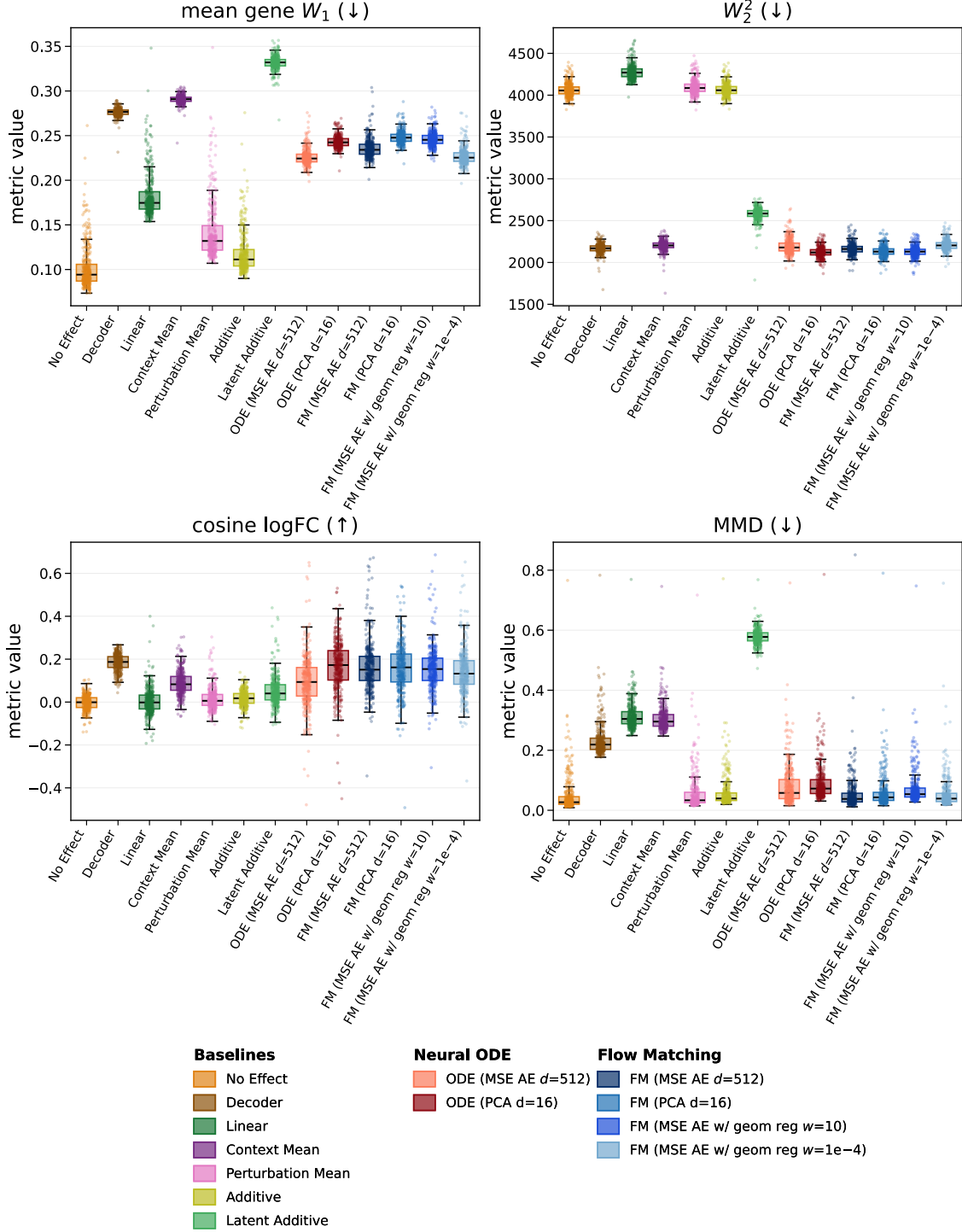


Figure 5.1: Context generalization performance across multiple metrics, baselines, and models. MMD is computed in PCA space (50 components), MSE, CosineLogFC, and W_2^2 are computed in log1p space after subsetting to HVGs and DEGs. Each metric is computed per perturbation-dose combination on the held-out perturbations for the K562 cell type. $\bar{W}_1$ is averaged over genes.

FM + ODE performance vs PCA dimensionality

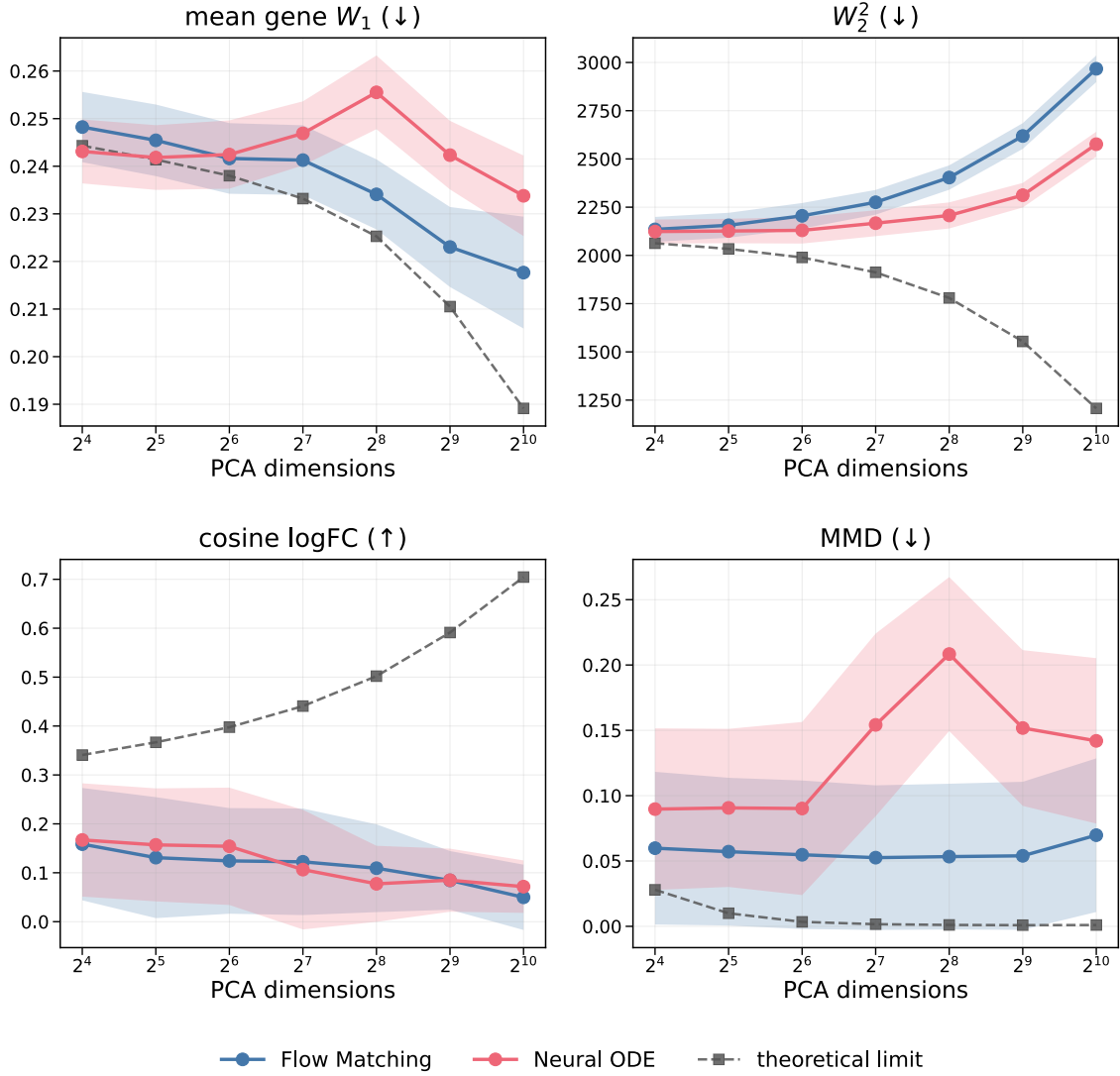


Figure 5.2: Performance of Flow Matching (FM) and Neural ODE (ODE) models in PCA space as dimension varies, compared to the computed upper bound of flow model performance on 4 metrics: Figure of four canonical perturbation prediction metrics, $\bar{W}_1$, W_2^2 , Cosine LogFC, and MMD (in 50-D PCA space), for standard held-out test split, metrics other than MMD computed in HVG+DEG ambient gene space as standard. The latent-space reconstruction limit is the result of evaluating a prediction of projecting the ground truth perturbed cells into the PCA space and projecting back to the ambient space. Shaded area represents $\pm$ STD, when grouping by dose and perturbation. FM and ODE models are in the PCA-space of the listed dimension. FM model is trained without OT coupling. Neural ODE is trained with the full loss (OT + energy + density). Results are for Sciplex3 dataset, held out K562 perturbations. PCA fit to training data only.

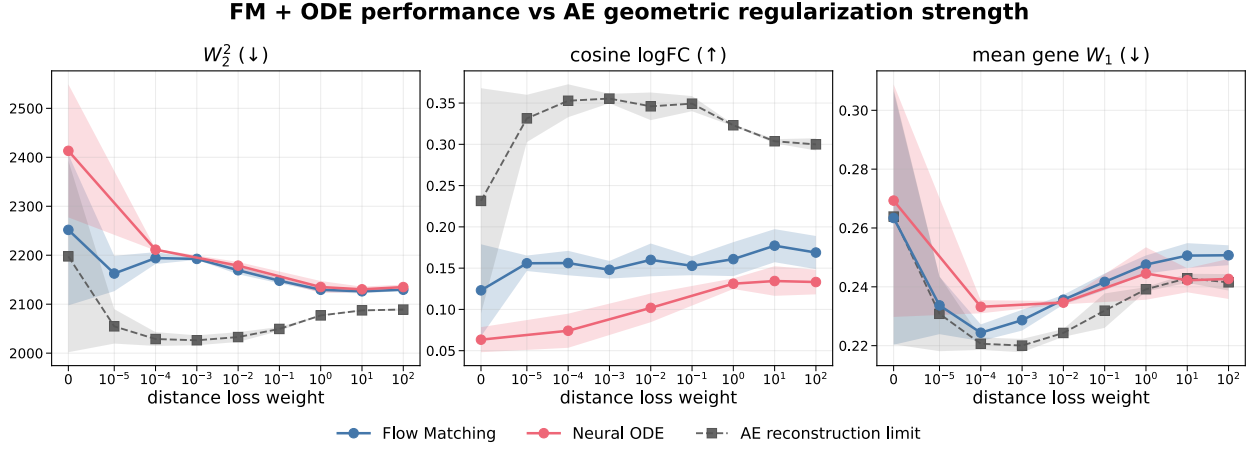


Figure 5.3: Performance of flow matching and Neural ODE models trained in latent space of a vanilla (MSE loss) autoencoder as a function of geometric regularization strength. The shaded region represents the $\pm$ std across seeds, where each seed determines train-val-test split of perturbations. All seeds use K562 as the underrepresented context, where which of 50% of perturbations are held-out is determined by seed. The autoencoder used is a vanilla autoencoder, trained with MSE loss comparing to log1p expression, with varying levels of geometric regularization strength, described in 4.4. Data corresponds to table 5.2. Gray squares with gray dashed lines represent the latent-space reconstruction limit of the metric in the latent space of the autoencoder, where predicted perturbed cells are reconstructed ground truth perturbed cells.

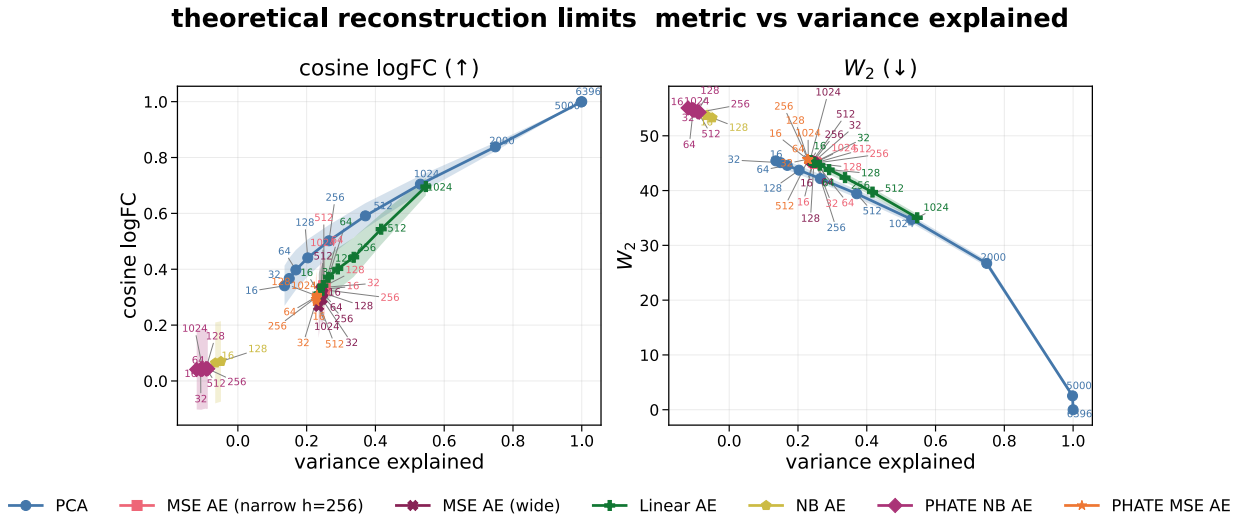


Figure 5.4: Latent-space reconstruction ceiling of Cosine LogFC (left) and W_2 (right) metrics compared to variance explained of different autoencoder models, with PCA as a baseline. Metrics and variance explained computed on held-out perturbations for K562 cells in Sciplex3 dataset, predicting perturbed cells as reconstruction of ground truth perturbed cells through AE/PCA. The shaded regions in each plot represent $\pm$ STD grouped by perturbation and dose. The NB autoencoder does not have geometric regularization whereas the PHATE autoencoder is a negative binomial (PHATE NB AE) or vanilla (PHATE MSE AE) autoencoder with geometric regularization.

6 Discussion

6.1 PCA/AE Bottleneck

PCA projects the data onto a linear k -dimensional hyperplane, whereas the non-linear activations of autoencoders allow it to learn a curved manifold. When we add geometric regularization to an autoencoder by introducing a distance matching loss, it trades off some amount of reconstruction flexibility for supporting stricter geometry of the autoencoder latent space. By matching Euclidean distances in the latent space with PHATE-derived manifold distances in processed expression space, geometric regularization encourages a latent space that we theorize is better suited for generative flow models. When included, the Negative Binomial aspect of the autoencoder ensures that the information lost in the bottleneck is primarily technical noise rather than biological signal, making the compressed latent space richer with biological information, rather than needing to capture irrelevant technical artifacts. In practice, we find that the Negative Binomial decoder degrades performance due to a mismatch between the space metrics are computed in and the loss is computed in (log1p vs counts space).

When considering the bottleneck from autoencoder (or dimensionality reduction generally) on downstream metric performance of generative flow models, discussed in Section 5, our experimental results support the theoretical finding that PCA is optimal for minimizing the latent-space reconstruction floor of W_2 for a given amount of cumulative variance explained, but not necessarily for dimensionality of the latent space. Because autoencoders produce non-linear transformations in their encoders and decoders, due to the non-linearity of the activation function used, they can have better latent-space reconstruction limits on metrics like W_2 as well as a higher cumulative variance explained in the same number of latent dimensions.

These findings motivate a careful consideration of the dimensionality reduction and dimensionality of the latent space used, if training in a latent space is to be done at all, when designing generative flow models for perturbation prediction or other single cell modeling. We suggest the latent-space reconstruction limit formulation of model performance of predicted perturbation cells as encoding then decoding ground truth perturbed cells, as also mentioned in Klein et al. [17] during model design and optimization. Without this in mind, effort could be wasted optimizing a generative flow model trained in a latent space where the bottleneck imposed results in the latent-space reconstruction limits of flow model performance that is already worse or close to the desired performance on a metric. We find that recent papers neglect or underemphasize these upper bounds on model performance, potentially wasting effort.

While we see strong performance of generative flow models trained in AE, with only MSE loss latent space, on metrics that only consider the mean per-gene, like Cosine LogFC, MSE loss for autoencoders has many disadvantages when it comes to predicting accurate per-gene distributional effects of perturbations.

GLFM optionally uses a negative binomial (NB) decoder as part of the autoencoder trained with negative log likelihood loss (NLL). This was motivated by [7, 25] to produce more realistic single-gene distributions of counts data. We find experimentally that on a

per-gene distributional view, the NB autoencoder produces qualitatively better distributions. Unfortunately, this autoencoder setup performs worse than a standard autoencoder trained with MSE loss. One reason for this, we suspect, is that the NB NLL loss has gradients that scale with the raw count magnitude, the $x \log \mu$ term in the NLL loss formula in Section 4.4. This means that high-count housekeeping genes dominate the gradient and the signal from low-count differentially expressed genes (DEGs) can be drowned out. When we train an autoencoder with MSE loss in log1p space, this treats all the genes equally, optimizing directly in the evaluation space. To clarify, we typically train in an "ambient" space where we first select the top DEGs for each perturbation as well as highly variable genes (HVG) across perturbations, then we normalize so each cell has the same sum of counts of 10k, then apply log1p transformation ($\log(1 + \text{normalized count})$). Since the NB autoencoder must compute its loss against raw counts, this creates a mismatch between the training objective and the evaluation space for the NB autoencoder. The MSE autoencoder avoids this mismatch by training and evaluating in the same log1p space. Additionally, we show in A that the MSE reconstruction loss is an upper bound on the W_2 distance between the ground-truth and reconstructed distributions. This means that training an autoencoder with MSE loss directly minimizes an upper bound on W_2 . This coupling between training objective and evaluation metric is a significant advantage of the MSE autoencoder over the NB NLL objective which does not have a similar direct connection. We propose trying to scale the loss for the NB autoencoder per-gene by something like average logFC across perturbations to emphasize accurate predictions on genes of interest and not just high-count housekeeping genes.

We found that using softmax at the end of the negative binomial decoder of an autoencoder systematically overpredicts expression. This is because the output of softmax can't be zero, while a significant proportion (88%) of raw counts are zero in the SciPlex3 dataset. The motivation of applying a softmax across genes is to produce a probability distribution over genes, which is then multiplied by the library size to get the mean parameter for each per-gene negative binomial distribution. When switching to computing an exponential of the log-rate independently for each gene (Section 4.4.1), each μ_g can approach zero for gene with strongly negative logits, which more correctly represents the high fraction of zeros in the count entries in SciPlex3. In the end, we ended up not using a negative binomial decoder for most tests. While negative binomial decoders produced per-gene distributions that visually matched the ground truth much better, the flow matching models trained in their latent spaces performed worse on most or all metrics, due to the training objective and evaluation metric mismatch of the negative binomial decoder.

6.2 What Geometric Regularization to Use?

The core motivation for using a geometric regularization on the latent space of an autoencoder for perturbation prediction is to simplify or better align the training objective of a flow matching or neural ODE model with the geometry of the latent space the model is trained in. If we have a latent space that is highly "curved", as defined for instance by high curvature of the Riemannian metric induced by the decoder, then the linear interpolations used during flow matching training will not be at logical places in the latent space. This is true even when not using a latent space, as the interpolation between a control and perturbed cell is not guaranteed or even necessarily likely to be close to a cell in the data,

i.e. on the data manifold. One way to work around this is to use more explicitly the Riemannian metric when performing flow matching training, like in Chen and Lipman [26] but this requires forward simulation of the ODE and defining an appropriate premetric on the data manifold, adding complexity compared to just flattening the latent space.

One approach of geometric regularization is what is done in FlatVI [7], which is to regularize the metric induced by the autoencoder decoder to be a scaled isometry (αI) everywhere. This forces straight lines in the latent space of the autoencoder to decode to geodesics on the (statistical) manifold in the decoded space. CFlows/GAGA [19, 6] geometric regularization match pairwise PHATE distances between cells on the ambient data manifold to the corresponding euclidean distances in the latent space of an autoencoder. We chose to do the distance preservation style loss of CFlows/GAGA since it is simpler to compute, not requiring a Jacobian computation through the NB decoder.

We hypothesize that PCA acts as a form of implicit regularization for the flow matching problem, not through imposing geometric structure, but by restricting the learning problem to a linear subspace which is inherently denoised. Because PCA is linear, the linear interpolations used during flow matching training will correspond to linear trajectories in the linear subspace of the ambient data. PCA does not flatten the data manifold or make linear trajectories more meaningful in ambient space, but it does strip away noise and simplify the learning task to axes of the data that contain signal, reducing the effective dimensionality of the vector field that needs to be learned. This may explain why flow matching in lower dimensional PCA space outperforms flow matching in higher dimensional PCA space on several metrics (Figure 5.2), despite the improved bounds on metrics at higher d .

6.3 Limitations

One limitation of this paper is that we primarily train on one dataset, Sciplex3 [13]. This dataset only includes 3 immortalized human cancer cell lines (K562, A549, MCF7), which limits the scope of the results of the method. We aim to expand our results by training on other context generalization datasets, like mentioned in PerturBench [12].

Another limitation is that we are explicitly ignoring the trajectory-structure induced by the drug doses and instead treat dose as a perturbation covariate. Existing methods (see Section 3) model trajectories in single cell data and other data sets based on some ordinal progression, like development of embryoid bodies [27], but we do not use the distribution of single cell data at different drug doses as marginals in this method. This is mostly because not all drug perturbation datasets contain data at different doses for the same drug.

Our standard evaluation is on one split of held-out perturbations for the K562 cell line in the Sciplex3 dataset, rather than doing cross-validation. The uncertainty estimates shown in tables are variance across seeds, not variance across split choice. We generally did not perform cross-validation for speed of model development. We do report cross-validation, pinning K562 as the held-out cell type, but using different splits of held-out perturbations when reporting geometric regularization strength results in 5.3 and 5.2.

We also chose to not benchmark against SOTA models for time considerations. Unfortunately, there are still not standardized benchmarks for perturbation prediction under-represented context generalization (same as covariate transfer).

We chose to integrate with PerturBench [12] as a standard benchmark framework because it includes careful consideration of metrics that are more relevant to biology than many other papers. For example, it includes a DEG overlap metric, which we use in Appendix Table 3, which measures the overlap between true differentially expressed gene and predicted differentially expressed genes. This is relevant to downstream tasks because of the applicability of differential expression as a reliable indicator of actual perturbation effects, but is not limited to statistically significant differentially expressed genes, which confounds results.

We discovered we have test data leakage when computing differentially expressed genes and highly variable genes. This likely does not significantly affect results. We end up with a set of 6,400 genes for the standard split (K562), which is a moderate portion of the total number of genes.

We choose to subset the dataset to 100k cells and include all doses for each perturbation. This results in around 30-80 cells for each drug+dose combination on the Sciplex3 dataset, which could harm the reliability of distributional metrics. It may have been wiser to use all cells of the highest dose, dropping lower dose perturbations since those are likely to have smaller effects.

We briefly tried using pretrained molecular embeddings from MolFormer-XL [28] instead of learned embeddings for the condition encoder and found that it did not significantly change performance. The performance for one-hot molecule embeddings compared to MolFormer-XL on W1 (0.229 vs 0.232) and Cosine LogFC (0.132 vs 0.13) was indistinguishable. This is not entirely unexpected given our evaluation protocol, since held-out drugs appear in training data for other cell lines, our model is able to learn to some extent how those drugs act in human cells. We discuss using pretrained molecular embeddings for completely unseen drug perturbation prediction in 8, where molecular embeddings or some other prior knowledge about the drugs would be required.

7 Conclusions

Problem Recap and Approach

We study the task of predicting single-cell transcriptional responses to drug perturbations in underrepresented contexts. This task has direct value in the pharmaceutical and clinical settings and could speed up drug development through in-silico testing of compounds. Accurate prediction of heterogeneous responses to drugs in underrepresented contexts could accelerate drug discovery by allowing for cheaper screening of large sets of compounds by imputing results from the same drugs tested on other cell lines, among other uses. We introduce GLFM, which trains generative flow models (Flow Matching and Neural ODEs) in the latent space of autoencoders, adding geometric regularization via distance-preservation loss to preserve structure in the autoencoder latent space. We compare GLFM against vanilla autoencoders, PCA, and standard baselines.

Empirical Findings

We found that the choice of autoencoder reconstruction loss and latent dimensionality has the largest effect on downstream flow matching and Neural ODE model performance in the latent space of the autoencoder model. We also found that geometric regularization on the autoencoder improves generative flow model performance and stabilizes autoencoder training, but that different levels of regularization are optimal for different metrics in the scenario we tested.

Counterintuitively, higher dimensional PCA space improves the latent-space reconstruction ceiling on metrics like W_2^2 and Cosine LogFC, but we find that FM and Neural ODE model performance is better in lower dimensional PCA space. We also find that autoencoders trained with MSE loss are particularly well suited for W_2^2 because MSE reconstruction loss is an upper bound on W_2^2 on the empirical training distribution (different from the distribution we compute W_2^2 on in practice, the empirical distribution of held-out perturbed cells).

For geometric regularization, we show that at a fixed latent dimension of an autoencoder, adding geometric regularization via a distance-preservation loss (H.1) improves performance across the three metrics tested, $\bar{W}_1$, W_2^2 and Cosine LogFC, when training both Flow Matching and Neural ODE models in the latent space of the autoencoder. Yet, the optimal distance loss weight for $\bar{W}_1$ in this setup is lower than for W_2^2 and Cosine LogFC, where increasing the distance loss weight further continues to improve flow model performance on those metrics.

Across all metrics, simple baselines remain very competitive. GLFM-style generative flow models are most useful on metrics that are sensitive to perturbation direction and multivariate distributional structure, while no-effect/additive baselines dominate marginal distribution metrics.

Theoretical Framing

Theoretically, under certain conditions, we have:

- Euclidean distances in the latent space of a geometrically regularized autoencoder with distance loss of zero (see H.1) approximate geodesic distances on the ambient data manifold

- Neural ODE model trained with energy loss and sufficiently strong static-OT penalty on the target marginal recovers the dynamic OT velocity field (see Theorem 3 from Huguet et al. [4], Section 4.7.2)
- Flow Matching training with (x_0, x_1) pairs from the static OT coupling recovers the dynamic OT velocity field as the conditional path width $\sigma \rightarrow 0$ (see OT-CFM Proposition 3.4 [29])

Together this suggests that training a flow model in a flattened latent space with OT coupling/penalty approximates dynamic OT along data-manifold geodesics. In practice, we find that the dimensionality and variance explained of the latent space has a larger impact than geometric regularization does, even though only geometric regularization aligns the latent geometry with the conditions above.

Implications & Takeaways

We suggest that latent-space reconstruction ceiling analysis become a standard design step when performing single cell modeling in latent spaces. When possible, one should derive mathematical limits to model performance based on the dimensionality reduction method (e.g., PCA, MSE loss Autoencoder). We also note that the choice of training versus evaluation space matters for performance on different metrics. One remaining open question both theoretically and experimentally is whether learning the dynamic OT vector field for geodesics on the data manifold is optimal for downstream metric performance.

We see that matching per-gene cell distributions (e.g., by using a negative binomial decoder) degrades model performance when compared to a vanilla autoencoder, despite a better visual fit at the gene level. This suggests that even if a model reconstructs accurate single-cell count data and approximates geodesics on the data manifold, it may not be optimal for downstream-relevant metrics like Cosine LogFC.

Limitations

Several limitations constrain the scope of these findings:

- results only reported on one dataset (Sciplex3)
- non-CV evaluation for main experiments except for geometric regularization strength sweep
- test data included when computing HVGs and DEGs for subsetting
- trajectory structure of doses ignored

See Section 6.3 for more details.

Future Directions

In Chapter 8 we identify many different directions for future geometry-informed single cell modeling using generative flows. The main direction we propose involves generally designing generative flow models to optimize for specific metrics of interest, by carefully considering the loss functions and space of training and metric evaluation. Building on recent developments in the generative model field in the past few years (see [21, 30, 31] as examples) seems underexplored and worthwhile for single cell modeling, as theoretical frameworks start to develop into concrete advances in the space.

8 Future Work

One area we are currently exploring is applying geometry-informed ground costs to flow models on $\mathbb{S}_+^d$, the positive orthant of the d -hypersphere, for modeling data as a probability distribution over categories, such as single cell data. Davis et al. [32] in Fisher Flow present another simplification for flow modeling by reparameterizing discrete data to the positive orthant. They use geodesics of $\mathbb{S}_+^d$ to perform Riemannian OT, while we propose using OT with ground costs integrating distances derived from the data manifold, like PHATE distances or spectral distances (e.g., Biharmonics) [31].

Another area of future work is to extend the framework of generative flows in the latent space of a geometrically-regularized autoencoder and integrate with pretrained molecular embeddings such as ChemBERT [33] to predict the response to perturbations not seen in the training data. This requires pretrained or learned representations of molecular or other perturbations to function, as our current framework is unable to extend to novel (unseen) perturbations.

We also propose a new way of designing loss functions for autoencoders to optimize for a specific metric or set of metrics for generative flow models, following from our results showing that minimizing MSE when training an autoencoder optimizes for the minimal latent-space reconstruction floor on W_2 metric. Similarly, we hypothesize that there are autoencoder loss functions that will maximize the latent-space reconstruction limits of other metrics for generative flow models, such as DEG Recall, MMD in PCA space, or Cosine LogFC rank (see [12]). Similarly, we propose designing a combination of autoencoder and generative flow model together to mathematically optimize for a specific metric. For example, designing some loss function for an autoencoder which can be shown to directly improve flow matching performance. Another area of interest is using other dimensionality reduction techniques that could exhibit different bottleneck behavior than PCA (either theoretically or experimentally) while remaining simpler to train and easier to analyze than full autoencoder, but many nonlinear dimensionality techniques lack a natural inverse mapping back to ambient gene-space, which reduces their applicability for modeling perturbation effects on a per-gene level.

Finally, there have recently been large single cell perturbation datasets, like Tahoe-100M [34], which greatly increase the dataset size for single cell perturbation data (around 10x larger than the next largest dataset at time of release). This presents the opportunity to scale up generative models from smaller, experimental models, to closer to foundational-model scale. Existing models have already been trained on these larger datasets (see [35]) but more work needs to be done for geometry-informed single cell modeling on 100M-cell scale data.

Bibliography

- [1] *Virtual Cell Challenge*. Virtual Cell Challenge. URL: <https://virtualcellchallenge.org/> (visited on 03/02/2026).
- [2] Artur Szałata et al. “A benchmark for prediction of transcriptomic responses to chemical perturbations across cell types”. In: ().
- [3] Yusuf H. Roohani et al. “Virtual Cell Challenge: Toward a Turing test for the virtual cell”. In: *Cell* 188.13 (June 2025), pp. 3370–3374. ISSN: 00928674. DOI: 10.1016/j.cell.2025.06.008. URL: <https://linkinghub.elsevier.com/retrieve/pii/S0092867425006750> (visited on 02/13/2026).
- [4] Guillaume Huguet et al. *MIOFlow: Manifold Interpolating Optimal-Transport Flows for Trajectory Inference*. Nov. 3, 2022. DOI: 10.48550/arXiv.2206.14928. arXiv: 2206.14928[cs]. URL: <http://arxiv.org/abs/2206.14928> (visited on 10/11/2025).
- [5] Xingzhi Sun et al. “CFlow: Revealing dynamic temporal trajectories and underlying regulatory networks with Cflows”. In: ().
- [6] Xingzhi Sun et al. *GAGA: Geometry-Aware Generative Autoencoders for Warped Riemannian Metric Learning and Generative Modeling on Data Manifolds*. Apr. 7, 2025. DOI: 10.48550/arXiv.2410.12779. arXiv: 2410.12779[cs]. URL: <http://arxiv.org/abs/2410.12779> (visited on 10/22/2025).
- [7] Alessandro Palma et al. *FlatVI: Enforcing Latent Euclidean Geometry in Single-Cell VAEs for Manifold Interpolation*. July 15, 2025. DOI: 10.48550/arXiv.2507.11789. arXiv: 2507.11789[cs]. URL: <http://arxiv.org/abs/2507.11789> (visited on 01/13/2026).
- [8] Yaron Lipman et al. *Flow Matching Guide and Code*. Dec. 9, 2024. DOI: 10.48550/arXiv.2412.06264. arXiv: 2412.06264[cs]. URL: <http://arxiv.org/abs/2412.06264> (visited on 12/11/2025).
- [9] Ricky T. Q. Chen et al. *Neural Ordinary Differential Equations*. Dec. 14, 2019. DOI: 10.48550/arXiv.1806.07366. arXiv: 1806.07366[cs]. URL: <http://arxiv.org/abs/1806.07366> (visited on 02/02/2026).
- [10] Amir Gholami, Kurt Keutzer, and George Biros. *ANODE: Unconditionally Accurate Memory-Efficient Gradients for Neural ODEs*. July 1, 2019. DOI: 10.48550/arXiv.1902.10298. arXiv: 1902.10298[cs]. URL: <http://arxiv.org/abs/1902.10298> (visited on 04/19/2026).
- [11] Mohammad Lotfollahi et al. “CPA: Predicting cellular responses to complex perturbations in high-throughput screens”. In: *Molecular Systems Biology* 19.6 (June 12, 2023). ISSN: 1744-4292, 1744-4292. DOI: 10.15252/msb.202211517. URL: <https://www.embopress.org/doi/10.15252/msb.202211517> (visited on 07/23/2025).
- [12] Yan Wu et al. *PerturBench: Benchmarking Machine Learning Models for Cellular Perturbation Analysis*. Oct. 24, 2025. arXiv: 2408.10609[cs]. URL: <http://arxiv.org/abs/2408.10609> (visited on 12/17/2025).

- [13] Sanjay R. Srivatsan et al. “Massively multiplex chemical transcriptomics at single-cell resolution”. In: *Science* 367.6473 (Jan. 3, 2020), pp. 45–51. ISSN: 0036-8075, 1095-9203. DOI: 10.1126/science.aax6234. URL: <https://www.science.org/doi/10.1126/science.aax6234> (visited on 03/04/2026).
- [14] Haotian Cui et al. “scGPT: toward building a foundation model for single-cell multi-omics using generative AI”. In: *Nature Methods* 21.8 (Aug. 2024), pp. 1470–1480. ISSN: 1548-7091, 1548-7105. DOI: 10.1038/s41592-024-02201-0. URL: <https://www.nature.com/articles/s41592-024-02201-0> (visited on 07/14/2025).
- [15] Yuansong Zeng et al. *CellFM: a large-scale foundation model pre-trained on transcriptomics of 100 million human cells*. June 6, 2024. DOI: 10.1101/2024.06.04.597369. URL: <http://biorxiv.org/lookup/doi/10.1101/2024.06.04.597369> (visited on 07/14/2025).
- [16] Yaron Lipman et al. *Flow Matching for Generative Modeling*. Feb. 8, 2023. DOI: 10.48550/arXiv.2210.02747. arXiv: 2210.02747[cs]. URL: <http://arxiv.org/abs/2210.02747> (visited on 10/26/2025).
- [17] Dominik Klein et al. “CellFlow enables generative single-cell phenotype modeling with flow matching”. In: ().
- [18] Manzil Zaheer et al. *Deep Sets*. Version Number: 3. 2017. DOI: 10.48550/ARXIV.1703.06114. URL: <https://arxiv.org/abs/1703.06114> (visited on 03/05/2026).
- [19] Xingzhi Sun et al. “CFlows: Revealing dynamic temporal trajectories and underlying regulatory networks with Cflows”. In: ().
- [20] Romain Lopez et al. “Deep generative modeling for single-cell transcriptomics”. In: *Nature Methods* 15.12 (Dec. 2018), pp. 1053–1058. ISSN: 1548-7091, 1548-7105. DOI: 10.1038/s41592-018-0229-2. URL: <https://www.nature.com/articles/s41592-018-0229-2> (visited on 12/13/2025).
- [21] Yiyang Lu et al. *Pixel Mean Flows: One-step Latent-free Image Generation with Pixel Mean Flows*. Feb. 5, 2026. DOI: 10.48550/arXiv.2601.22158. arXiv: 2601.22158[cs]. URL: <http://arxiv.org/abs/2601.22158> (visited on 02/09/2026).
- [22] Yang Song et al. *Score-Based Generative Modeling through Stochastic Differential Equations*. Feb. 10, 2021. DOI: 10.48550/arXiv.2011.13456. arXiv: 2011.13456[cs]. URL: <http://arxiv.org/abs/2011.13456> (visited on 04/18/2026).
- [23] Gerold Csendes, Kristóf Z. Szalay, and Bence Szalai. *Benchmarking a foundational cell model for post-perturbation RNAseq prediction*. Oct. 1, 2024. DOI: 10.1101/2024.09.30.615843. URL: <http://biorxiv.org/lookup/doi/10.1101/2024.09.30.615843> (visited on 10/08/2025).
- [24] Aaron Wenteler et al. *PertEval-scFM: Benchmarking Single-Cell Foundation Models for Perturbation Effect Prediction*. Oct. 3, 2024. DOI: 10.1101/2024.10.02.616248. URL: <http://biorxiv.org/lookup/doi/10.1101/2024.10.02.616248> (visited on 08/01/2025).
- [25] Romain Lopez et al. “scVI: Deep generative modeling for single-cell transcriptomics”. In: *Nature Methods* 15.12 (Dec. 2018), pp. 1053–1058. ISSN: 1548-7091, 1548-7105. DOI: 10.1038/s41592-018-0229-2. URL: <https://www.nature.com/articles/s41592-018-0229-2> (visited on 01/22/2026).

- [26] Ricky T. Q. Chen and Yaron Lipman. *Riemannian Flow Matching: Flow Matching on General Geometries*. Feb. 26, 2024. DOI: 10.48550/arXiv.2302.03660. arXiv: 2302.03660[cs]. URL: <http://arxiv.org/abs/2302.03660> (visited on 01/13/2026).
- [27] Alexander Tong et al. *TrajectoryNet: A Dynamic Optimal Transport Network for Modeling Cellular Dynamics*. July 26, 2020. DOI: 10.48550/arXiv.2002.04461. arXiv: 2002.04461[stat]. URL: <http://arxiv.org/abs/2002.04461> (visited on 12/15/2025).
- [28] Jerret Ross et al. “Large-scale chemical language representations capture molecular structure and properties”. In: *Nature Machine Intelligence* 4.12 (Dec. 21, 2022), pp. 1256–1264. ISSN: 2522-5839. DOI: 10.1038/s42256-022-00580-7. URL: <https://www.nature.com/articles/s42256-022-00580-7> (visited on 01/25/2026).
- [29] Alexander Tong et al. *OT-CFM: Improving and generalizing flow-based generative models with minibatch optimal transport*. Mar. 11, 2024. DOI: 10.48550/arXiv.2302.00482. arXiv: 2302.00482[cs]. URL: <http://arxiv.org/abs/2302.00482> (visited on 12/15/2025).
- [30] Oscar Davis et al. “Generalised Flow Maps for Few-Step Generative Modelling on Riemannian Manifolds”. In: (2026).
- [31] Ricky T. Q. Chen and Yaron Lipman. *Flow Matching on General Geometries*. Feb. 26, 2024. DOI: 10.48550/arXiv.2302.03660. arXiv: 2302.03660[cs]. URL: <http://arxiv.org/abs/2302.03660> (visited on 12/15/2025).
- [32] Oscar Davis et al. “Fisher Flow Matching for Generative Modeling over Discrete Data”. In: ().
- [33] Riya Singh et al. *ChemBERTa-3: An Open Source Training Framework for Chemical Foundation Models*. Aug. 3, 2025. DOI: 10.26434/chemrxiv-2025-4glrl-v2. URL: <https://chemrxiv.org/doi/full/10.26434/chemrxiv-2025-4glrl-v2> (visited on 01/25/2026).
- [34] Jesse Zhang et al. *Tahoe-100M: A Giga-Scale Single-Cell Perturbation Atlas for Context-Dependent Gene Function and Cellular Modeling*. Feb. 24, 2025. DOI: 10.1101/2025.02.20.639398. URL: <http://biorxiv.org/lookup/doi/10.1101/2025.02.20.639398> (visited on 08/01/2025).
- [35] Abhinav K Adduri et al. “STATE: Predicting cellular responses to perturbation across diverse contexts with State”. In: ().
- [36] Yusuf Roohani, Kexin Huang, and Jure Leskovec. “GEARS: Predicting transcriptional outcomes of novel multigene perturbations with GEARS”. In: *Nature Biotechnology* 42.6 (June 2024), pp. 927–935. ISSN: 1087-0156, 1546-1696. DOI: 10.1038/s41587-023-01905-6. URL: <https://www.nature.com/articles/s41587-023-01905-6> (visited on 07/14/2025).
- [37] Kevin R. Moon et al. “PHATE: Visualizing structure and transitions in high-dimensional biological data”. In: *Nature Biotechnology* 37.12 (Dec. 2019), pp. 1482–1492. ISSN: 1087-0156, 1546-1696. DOI: 10.1038/s41587-019-0336-3. URL: <https://www.nature.com/articles/s41587-019-0336-3> (visited on 07/16/2025).

9 Appendix

A Theoretical Derivations, Baselines, and Extra Definitions

Table 1: Summary of idealized PCA scaling laws as a function of cumulative variance explained CVE_k by the top- k PCA components. TV denotes total variance $\text{Tr}(\Sigma)$, G is the number of genes.

Metric	Limit	Type	Assumptions
Cosine LogFC	$\approx \sqrt{\text{CVE}_k}$	Approximation	First-order Taylor on ratio
W_2	$\sqrt{\text{TV}} \cdot \sqrt{1 - \text{CVE}_k}$	Exact	Gaussian data
$\overline{W}_1$	$\leq \sqrt{\frac{2\text{TV}}{\pi G}} \sqrt{1 - \text{CVE}_k}$	Upper bound	Gaussian gene marginals

A.1 Setup and Notation

In this appendix, let $\Sigma \in \mathbb{R}^{G \times G}$ be the population covariance of the mean-centered gene expression data, with eigendecomposition $\Sigma = V\Lambda V^\top$, where $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_G)$ sorted in descending order by eigenvalue. Let $V_k \in \mathbb{R}^{G \times k}$ contain the top- k eigenvectors and $P_k = V_k V_k^\top \in \mathbb{R}^{G \times G}$ be the orthogonal projection onto the top- k PCA subspace, expressed in the ambient gene-coordinate basis. We let $\Delta g = g_{\text{pert}} - g_{\text{ctrl}}$ be the change in gene expression and $\Delta \hat{g} = P_k \Delta g$ be its PCA reconstruction. Throughout the distributional derivations, PCA reconstruction is applied to centered data and the mean is added back so the original and reconstructed distributions have the same mean: $\hat{X}_k = \mu + P_k(X - \mu)$.

We use the following properties throughout:

- **Idempotency and symmetry:** $P_k^2 = P_k$ and $P_k^\top = P_k$ since P_k is an orthogonal projection
- **Commutativity with Σ :** P_k and Σ share an eigenbasis, so $P_k \Sigma = \Sigma P_k = P_k \Sigma P_k$. In particular, we use that $\Sigma_k := P_k \Sigma P_k$ simplifies to $\Sigma_k = P_k \Sigma$
- **Trace identities:** $\text{Tr}(\Sigma) = \sum_{j=1}^G \lambda_j$ (total variance) and $\text{Tr}(P_k \Sigma) = \sum_{i=1}^k \lambda_i$ so the cumulative variance explained is $\text{CVE}_k = \text{Tr}(P_k \Sigma) / \text{Tr}(\Sigma)$

The W_2 and $\overline{W}_1$ derivations assume the data is Gaussian, which allows for closed-form expressions using the Bures metric for W_2 and the 1D Gaussian formula for $\overline{W}_1$. The Cosine LogFC derivation does not require the data to be Gaussian, but uses a first-order Taylor approximation of the expectation of a ratio, which is justified by the concentration of $\|\Delta g\|^2$ in high dimensions. These properties are specific to PCA and do not generally hold for nonlinear dimensionality reduction methods.

A.2 Idealized PCA Scaling for Cosine LogFC

We want to derive an idealized PCA scaling law for the Cosine LogFC metric (see F.3) as a function of the cumulative variance explained in a PCA projection. We show below that $\mathbb{E}[\text{Cosine LogFC}] \approx \sqrt{\text{CVE}_k}$ where CVE_k is the cumulative variance explained of a PCA projection sub-setting to the first k (sorted by largest eigenvalue first) principal components.

We start with the base formula for Cosine LogFC defined in F.3 and expand. To recap, Cosine LogFC is the cosine similarity between the ground truth log fold-change vector Δg , comparing the ground truth perturbed to ground truth control cells, and the predicted log fold-change vector $\Delta \hat{g}$, comparing the predicted perturbed to ground truth control cells. Now to expand:

$$\text{Cosine LogFC} = \frac{\Delta \hat{g} \cdot \Delta g}{\|\Delta \hat{g}\| \cdot \|\Delta g\|} = \frac{(P_k \Delta g)^\top \Delta g}{\|P_k \Delta g\| \cdot \|\Delta g\|} = \frac{\Delta g^\top P_k \Delta g}{\|P_k \Delta g\| \cdot \|\Delta g\|}$$

We can simplify the numerator:

$$\Delta g^\top P_k \Delta g = \Delta g^\top P_k^\top P_k \Delta g = (P_k \Delta g)^\top (P_k \Delta g) = \|P_k \Delta g\|^2$$

So we can collapse Cosine LogFC to:

$$\text{Cosine LogFC} = \frac{\|P_k \Delta g\|^2}{\|P_k \Delta g\| \cdot \|\Delta g\|} = \frac{\|P_k \Delta g\|}{\|\Delta g\|} = \sqrt{\frac{\|P_k \Delta g\|^2}{\|\Delta g\|^2}}$$

Now we show $\mathbb{E} \left[\frac{\|P_k \Delta g\|^2}{\|\Delta g\|^2} \right] = \frac{\sum_{i=1}^k \lambda_i}{\sum_{j=1}^G \lambda_j} = \text{CVE}_k$ below, by first order Taylor approximation and simplification of the numerator and denominator.

Using a first-order Taylor approximation, assuming concentration of the square norm $\|\Delta g\|^2$ in high dimensions, to approximate the expectation of the ratio, we can split the expectation:

$$\mathbb{E} \left[\frac{\|P_k \Delta g\|^2}{\|\Delta g\|^2} \right] \approx \frac{\mathbb{E}[\|P_k \Delta g\|^2]}{\mathbb{E}[\|\Delta g\|^2]}$$

Now, simplifying the numerator and denominator independently, with the numerator simplification first:

$$\mathbb{E}[\|P_k \Delta g\|^2] = \mathbb{E}[\Delta g^\top P_k \Delta g] = \mathbb{E}[\text{Tr}(P_k \Delta g \Delta g^\top)] = \text{Tr}(\mathbb{E}[P_k \Delta g \Delta g^\top]) = \text{Tr}(P_k M_\Delta)$$

where $M_\Delta = \mathbb{E}[\Delta g \Delta g^\top]$. To simplify the denominator:

$$\mathbb{E}[\|\Delta g\|^2] = \mathbb{E}[\Delta g^\top \Delta g] = \mathbb{E}[\text{Tr}(\Delta g \Delta g^\top)] = \text{Tr}(\mathbb{E}[\Delta g \Delta g^\top]) = \text{Tr}(M_\Delta)$$

To connect this perturbation-effect quantity to CVE_k , we make an assumption that perturbation-effect energy is distributed across PCA directions similarly to ordinary expression variance. That is

$$\frac{\text{Tr}(P_k M_\Delta)}{\text{Tr}(M_\Delta)} \approx \frac{\text{Tr}(P_k \Sigma)}{\text{Tr}(\Sigma)}$$

This assumption is reasonable when perturbations move cells along dominant biological state axes, but could fail for sparse or pathway-specific perturbations lying in low-variance

directions. Using our simplification of the numerator and denominator from above with the same concentration assumption, $\mathbb{E}[\sqrt{X}] \approx \sqrt{\mathbb{E}[X]}$, giving:

$$\mathbb{E}[\text{CosineLogFC}] \approx \sqrt{\frac{\text{Tr}(P_k M_\Delta)}{\text{Tr}(M_\Delta)}} \approx \sqrt{\frac{\text{Tr}(P_k \Sigma)}{\text{Tr}(\Sigma)}} = \sqrt{\text{CVE}_k}$$

This matches the geometry of Cosine LogFC we see in practice, where Cosine LogFC is about the square root of cumulative variance explained.

A.3 Idealized PCA Scaling for W_2

For centered Gaussians with commuting covariance matrices, the Gaussian W_2 formula reduces to a sum over eigen-directions:

$$W_2^2(p, \hat{p}_k) = \sum_{i=1}^G \left(\sqrt{\lambda_i} - \sqrt{\lambda_i^{(k)}} \right)^2$$

where $\lambda_i^{(k)} = \lambda_i$ for $i \leq k$ and $\lambda_i^{(k)} = 0$ for $i > k$. This follows from the closed-form Gaussian W_2 distance

$$W_2^2 = \text{Tr} \left[\Sigma + \Sigma_k - 2 \left(\Sigma^{1/2} \Sigma_k \Sigma^{1/2} \right)^{1/2} \right]$$

since Σ and Σ_k are simultaneously diagonalizable in the PCA basis.

Therefore,

$$W_2^2(p, \hat{p}_k) = \sum_{i=k+1}^G \lambda_i = \text{Tr}(\Sigma)(1 - \text{CVE}_k)$$

Thus,

$$W_2(p, \hat{p}_k) = \sqrt{\text{Tr}(\Sigma)} \sqrt{1 - \text{CVE}_k}$$

This proves why empirically we see W_2 scale proportionally to the square root of $(1 - x)$. In reality, scRNA-seq data is not Gaussian, but can be modeled by a negative binomial distribution more accurately, but we still see this scaling in practice.

A.4 Idealized PCA Scaling for $\overline{W}_1$

We show that $\overline{W}_1$, the average W_1 distance between ground truth and prediction, averaged over genes, scales as $\sqrt{1 - \text{CVE}_k}$, the same as W_2 , under the same Gaussian assumption.

For each gene g , we set ρ_g to be the ground-truth marginal distribution for gene g and $\hat{\rho}_g$ to be the marginal of the PCA reconstruction for gene g . Assume that each gene marginal is Gaussian with the same mean (PCA is mean-centered).

For two 1D Gaussians with equal means, we have the explicit closed form for W_1 :

$$W_1(\mathcal{N}(\mu, \sigma_1^2), \mathcal{N}(\mu, \sigma_2^2)) = \sqrt{\frac{2}{\pi}} |\sigma_1 - \sigma_2|$$

The variance of gene g in the ground truth is Σ_{gg} and in the PCA reconstruction it is $(\Sigma_k)_{gg}$ where $\Sigma_k = P_k \Sigma$. Using that $(I - P_k) \Sigma$ is PSD (since $(I - P_k) \Sigma = (I - P_k) \Sigma (I - P_k)$)

via commutativity of P_k and Σ), we have $\Sigma_{gg} \geq (\Sigma_k)_{gg}$ for every gene. Then we sum to get:

$$\sum_{g=1}^G [\Sigma_{gg} - (\Sigma_k)_{gg}] = \text{Tr}(\Sigma) - \text{Tr}(\Sigma_k) = \text{TV}(1 - \text{CVE}_k)$$

Apply the Gaussian W_1 formula and use $(\sqrt{a} - \sqrt{b})^2 \leq a - b$ for $0 \leq b \leq a$:

$$\bar{W}_1 = \frac{1}{G} \sum_{g=1}^G W_1(\rho_g, \hat{\rho}_g) = \sqrt{\frac{2}{\pi}} \cdot \frac{1}{G} \sum_{g=1}^G \left| \sqrt{\Sigma_{gg}} - \sqrt{(\Sigma_k)_{gg}} \right|$$

Use Cauchy-Schwarz:

$$(\bar{W}_1)^2 \leq \frac{2}{\pi} \cdot \frac{1}{G} \sum_{g=1}^G \left(\sqrt{\Sigma_{gg}} - \sqrt{(\Sigma_k)_{gg}} \right)^2 \leq \frac{2}{\pi} \cdot \frac{1}{G} \sum_{g=1}^G [\Sigma_{gg} - (\Sigma_k)_{gg}] = \frac{2}{\pi G} \cdot \text{TV}(1 - \text{CVE}_k)$$

Take the square root:

$$\bar{W}_1 \leq \sqrt{\frac{2\text{TV}}{\pi G}} \cdot \sqrt{1 - \text{CVE}_k}$$

B Single Cell Dataset Preprocessing

To preprocess the single cell data, we first normalize the total counts for each cell to 10,000 and then apply a log1p transformation ($\log(1 + x)$), where log denotes natural logarithm. To compute highly variable genes (HVGs), we use scanpy's `highly_variable_genes` with flavor `seurat.v3` and batch key of cell type, which computes highly variable genes independently per cell type and then ranked by the median rank across cell types, where the top 2000 are chosen. This avoids selection of genes that just vary across cell types only and instead focuses on genes that vary within cell type, like due to perturbations. HVGs are computed on raw counts. To compute differentially expressed genes (DEGs) we use scanpy's `rank_gene_groups` after preprocessing (normalize total + log1p), grouping by perturbation and dose combination. This does a t-test to compare each perturbation group to control, matched by cell type, returning a score per gene. We then take the top 25 genes per perturbation group by highest score. We then do a union over the set of DEGs and HVGs to get around 6,400 genes for the SciPlex3 dataset. Our standard preprocessing does normalize total, log1p, then subsets to the roughly 6,400 genes from DEGs and HVGs.

C PCA Space for Training

When we train a generative flow model in PCA space, we compute PCs only on the training data, to prevent data leakage. We perform PCA after other normalization steps (normalize total, log1p, subsetting) and do not perform standard scaling on PC components. This results in PC1 having the largest variance, which is what the model sees. When we say that we use k -dimensional PCA space, this means we compute the top k principal components and use those as the dimensions of our new "latent space" (latent space used to refer to PCA space).

D Experimental Setup

For the autoencoder models, we train by default for 20 epochs, with a batch size of 256 and a learning rate of $1e-3$, with the AdamW optimizer. For the Flow Matching model, we train for 100 epochs by default, with a batch size of 1024, learning rate of $1e-4$, weight decay of 0.01, and AdamW optimizer. For the Neural ODE model, we train for 100 epochs with a batch size of 4000, learning rate of $1e-4$, weight decay of 0.01, and AdamW optimizer. For training the Flow Matching and Neural ODE models, we sample one perturbation condition at a time from the training set, sample a batch of `batch_size` of cells all from that perturbation (sampling with replacement) and perform a gradient update based on just this batch. We do condition-first sampling so that the OT coupling and loss operate within a single perturbation condition, which is required to make the velocity field regression task and OT coupling meaningful.

Training was performed on Yale’s HPC Bouchet cluster using a combination of RTX 6000 Pro, B200, L40s, and H200 GPUs. Training was always performed on a single GPU.

E Graph Based Models

One early class of single-cell models for perturbation prediction are graph-based models. The main model in this class is the GEARS model from Roohani, Huang, and Leskovec [36]. At the core of the GEARS model are two graph neural networks, one to learn baseline embeddings for genes specific to the training data and another to learn embeddings for gene perturbations. After obtaining the gene embedding for each gene as well as the perturbation embedding for each gene perturbation to predict in a set of perturbed genes (can be multiple), the perturbation embeddings for each perturbation are added to each gene embedding using element-wise addition. After this composition, a single MLP shared across all genes transforms the vector and then a gene-specific MLP is applied to decode out post-perturbation gene expression. The primary task of the GEARS model is to infer unseen combinations of gene perturbations, where the effect of perturbing the genes that make up the combination have usually been seen in other combinations of perturbations or individual gene perturbations. The GEARS model shows greatly improved performance over CPA, particularly when predicting the effects of a combination of two perturbations whose effects have not been seen anywhere in the training data.

This model is limited to predicting the effects of perturbations on genes and lacks an explicit mechanism to predict the effect of drug perturbations. Another limitation of this model is that it does not produce a distribution of predicted responses, rather the model produces a single scalar output for each gene for a given combination of perturbations. This is limiting because it does not model the heterogeneous response to cell perturbations and ignores the underlying heterogeneity of the cell population.

A critique of the setup of the GEARS model is that they use a direction-aware loss which is only superficially well motivated. This direction loss term penalizes perturbation effect predictions (perturbed expression minus control expression on a per-cell and per-gene level) that are the wrong sign (i.e., predicted decrease in expression of gene A when the actual expression increased in response to perturbation would be penalized). While

this seems well motivated, the penalty of this loss term is independent of the magnitude of the error or significance of the differential expression of the gene. In other words, if we treat the control cell and perturbed cell gene expression from gene A as both sampling from the same Gaussian distribution, this loss term would be non-zero 50% of the time even if the model predicted no change in gene expression and simply returned the gene expression of the control cell or the mean expression of that gene. There is a loss term which assigns a higher penalty to poor predictions for differentially expressed genes, the "autofocus" loss, which is most likely good enough for the purposes of this model. The paper shows that switching from the autofocus loss to a vanilla MSE loss greatly degrades model performance on MSE of the top 20 genes, as expected.

F Metrics

F.1 Maximum Mean Discrepancy

$$\text{MMD}^2(\rho, \nu) = \mathbb{E}[k(X, X')] + \mathbb{E}[k(Y, Y')] - 2\mathbb{E}[k(X, Y)]$$

where $X, X' \sim \rho$ and $Y, Y' \sim \nu$ are independent random vectors, defined the same as above. We use the Gaussian (RBF) kernel:

$$k(X, Y) = \exp\left(-\frac{\|X - Y\|_2^2}{2\sigma^2}\right)$$

Because it utilizes a non-linear kernel, MMD is sensitive to higher-order moments, so it can penalize models that correctly predict average gene expression but fail to capture biological heterogeneity of the cellular response to a drug. We compute MMD in a 50-dimensional PCA space, rather than comparing cells in the full ambient gene space. We project the cells onto the top 50 principal components computed only over the training data to prevent leakage.

F.2 (Squared) Wasserstein Distance (per-gene W_1 and multivariate W_2^2)

Wasserstein distance is defined as:

$$W_p(\rho, \nu) = \inf_{(X, Y) \sim \gamma \in \Pi(\rho, \nu)} \mathbb{E}_\gamma[\|X - Y\|^p]^{1/p}$$

where $X \sim \rho$ and $Y \sim \nu$ as above and Π is the set of all couplings of ρ and ν .

Wasserstein distance can be thought of as the minimum amount of work required to move probability density from one distribution to another, under a given penalty for distance moved (squared for W_2 distance and just distance moved for W_1 distance). We use W_2^2 as a metric to compare the true perturbed distribution and predicted perturbed distribution of cells for a given perturbation covariate combination (drug and dose), not computed per-gene.

We also use the average W_1 distance per-gene, then averaged over genes, again, for a given perturbation covariate combination:

$$\bar{W}_1 := \frac{1}{G} \sum_{g=1}^G W_1(\rho_g, \nu_g)$$

F.3 Cosine LogFC

The Cosine LogFC metric measures the cosine similarity between the LogFC vector comparing ground truth or predicted perturbed cell mean gene expressions to the control cell distribution mean gene expression. Let $\bar{x}'$, $\bar{x}_{\text{pert}}$, and $\bar{x}_{\text{ctrl}}$ be the vector of mean gene expressions for the predicted perturbed cells, ground truth perturbed cells, and (ground truth) control cell populations in log1p normalized space. We define the log-fold-change vector in log1p space, following standard single cell convention, as follows:

$$\Delta\hat{g} = \bar{x}' - \bar{x}_{\text{ctrl}}, \quad \Delta g = \bar{x}_{\text{pert}} - \bar{x}_{\text{ctrl}}$$

and report the metric

$$\text{CosineLogFC} = \frac{\Delta\hat{g} \cdot \Delta g}{\|\Delta\hat{g}\| \cdot \|\Delta g\|}$$

A perfect prediction gives Cosine LogFC of 1.

F.4 DEG Top- k Recall Metric

This metric is used in the PerturBench [12] comparison M.1 and measures the overlap (recall) between the top 50 DEG in the ground truth data (ground truth perturbed vs ground truth control) and the top 50 DEG in the predicted perturbed cells vs the ground truth control cells, taking the mean over all held-out perturbations. It uses absolute measurement of differential expression, not direction specific, so both up and down-regulated genes. This metric uses `scanpy.tl.rank_genes_groups` to compute differentially expressed genes, ranked by absolute t-score. If perturbation h has ground truth top 50 DEG set D_h^k and predicted top 50 DEG set $\hat{D}_h^k$, then DEG top- k recall is

$$\text{DEG}_k = \frac{1}{H} \sum_h \frac{|\hat{D}_h^k \cap D_h^k|}{k}$$

where H is the total number of perturbation+dose combinations (or generally perturbation covariate combinations).

G Baselines

G.1 No Effect Baseline

For the no effect baseline, we simply predict the input control cell x_{ctrl} as the perturbed cell x' , so $x' = x_{\text{ctrl}}$. If we randomly sample from control cells with matched covariates (e.g. cell type) for x_{ctrl} , this baseline will closely match the control cell distribution. For perturbations with very small effect or no significant effect at all, it is not possible to exceed the performance of this baseline in a significant way.

G.2 Linear Baseline

The linear baseline trains a linear layer to predict the change in gene expression based on the condition encoder encoding of the condition batch, then adds that to the control cell

gene expression to get the predicted post-perturbation gene expression. For control cell x_{ctrl} , prediction x' , linear layer F_{linear} and condition encoding $E_\phi(B'_x)$, the model predicts:

$$x' = x_{\text{ctrl}} + F_{\text{linear}}(E_\phi(B_{x'}))$$

In this case, $B_{x'}$ is the condition batch corresponding to the perturbed cell x' that we want to predict, not the control cell x_{ctrl} , since the drug and dose information are encoded in $B_{x'}$ and known during test-time, we want to predict the effect of the given perturbation for x' .

G.3 Decoder-Only Baseline

In the decoder-only baseline, we train an MLP to directly predict the post-perturbation gene expression using only the condition batch for each perturbed cell, passed through the condition encoder. For condition batch $B_{x'}$, condition encoder E_ϕ and decoder D_θ , and prediction x' , we have:

$$x' = D_\theta(E_\phi(B_{x'}))$$

This baseline disregards the control cell information, relying only on fitting to the perturbed cell gene expression. Since this model is not stochastic, it will predict the same post-perturbation gene expression when the condition batch is the same, resulting in deterministic predictions for a given covariate combination (perturbation, dose, cell line, etc.).

G.4 Additive Baseline

This learns a global effect of perturbing any cell and is, in practice, nearly identical to the No Effect baseline due to the low magnitude of effect of any given perturbation.

$$\hat{x} = x_{\text{ctrl}} + \delta$$

where $\delta = \mu_{\text{pert}} - \mu_{\text{ctrl}}$ is computed globally over all training data.

G.5 Perturbation Mean Baseline

In the perturbation mean baseline, we follow Adduri et al. [35]. For a held-out perturbation for a specific sample covariate combination (e.g., cell type), we predict the post-perturbation gene expression to be the average post-perturbation gene effect for that same perturbation across the training dataset, added to the gene expression for each individual control cell of the context we are generalizing to. The problem is set up such that each unseen perturbation in the sample context we are trying to generalize to has been seen in the training data in the different sample covariates (all other sample covariates in almost every case). This baseline is logically like assuming that each perturbation acts the same way no matter the cell context or control cell gene expression. To outperform the perturbation mean baseline, a model must learn cell-type specific effects of perturbations.

Given the mean gene expression of observed control cells $\mu_s^{\text{ctrl}} = \frac{1}{|\mathcal{S}_s|} \sum_{i \in \mathcal{S}_s} x^{(i)}$ and mean gene expression perturbed cells $\mu_{s,p}^{\text{pert}} = \frac{1}{|\mathcal{P}_{c,p}|} \sum_{i \in \mathcal{P}_{c,p}} x^{(i)}$ for sample covariates s , and perturbation covariates p , we compute the change in gene expression for a specific

(s, p) as $\delta_{s,p} = \mu_{s,p}^{\text{pert}} - \mu_s^{\text{ctrl}}$. This is averaged for each perturbation p over all sample covariates in the training data with that perturbation present, to get a global perturbation effect $\delta_p = \frac{1}{|\mathcal{S}_p|} \sum_{s \in \mathcal{S}_p} \delta_{s,p}$. For the underrepresented sample context t in the test/validation data and the perturbation p , the model predicts the post-perturbation gene expression for each control cell x^{ctrl} as $\hat{x} = x_t^{\text{ctrl}} + \delta_p$.

This notation is based on Adduri et al. [35], but the baseline is adapted to produce a distribution of predicted perturbed cells by adding the global perturbation effect to each control cell in the underrepresented context individually rather than to the average gene expression of the control cells in the underrepresented context. Metrics that explicitly or implicitly pseudo-bulk (average) gene expression within contexts, like Cosine LogFC, will be equivalent in this adapted method compared to the original, while distribution-specific baselines, like W_2 , can be better in the adapted version. For example if the perturbation has no effect, as the sample size of perturbed and control cells approaches infinity, the W_2 distance between ground truth perturbation distribution and predicted perturbation distribution for this method will approach zero, while for the original method in Adduri et al. [35] will approach the total variance.

G.6 Context Mean Baseline

In the context mean, we simply predict the post-perturbation gene expression in the underrepresented context as the mean gene expression across all cells in the underrepresented context. This does not produce a distribution of cells as a prediction, but rather a singular cell (repeated if needed) as the "average" cell in the context, for both control cells and cells in all perturbations seen in the training data for that cell type.

$$\mu_s = \frac{1}{|\mathcal{S}_s|} \sum_{i \in \mathcal{S}_s} x^{(i)}$$

H Autoencoder Losses

H.1 Distance Preservation Loss

Following Sun et al. [19] in CFlows, we use a distance loss to encourage preservation of manifold structure in the latent space of the autoencoder. We use PHATE [37] to compute pairwise distances between all cells in the training dataset using the diffusion-based method. Note that PHATE first projects the data in the ambient space (after we do normalization and filtering to selected genes, typically top DEG for each perturbation and HVG) using PCA, then computes PHATE distances in the PCA space. As discussed in Section 5, the bottleneck imposed by PCA improves the latent-space reconstruction limits of generative flow model performance, but we propose based on Moon et al. [37] that this bottleneck preserves enough geometric information about the data to result in meaningful PHATE coordinates and pairwise distances. To summarize the PHATE potential distance computation in the selected standard preprocessing pipeline:

raw counts $\rightarrow$ HVG + DEG subset $\rightarrow$ log1p normalize $\rightarrow$ PHATE (with internal PCA-100, knn=5, n_landmark=2000, t='auto')

Then, compute pairwise distances in this PHATE space.

This distance preservation loss penalizes the encoder E_ϕ of the autoencoder model during training and works in combination, but is independent of the decoder architecture

and loss used. Following the notation in Sun et al. [19]:

$$\mathcal{L}_{\text{dist}}(\phi) = \frac{1}{N} \sum_{i < j} e^{-\varsigma d(x_i, x_j)} (\|E_\phi(x_i) - E_\phi(x_j)\|_2 - d(x_i, x_j))^2$$

Where ς is a hyperparameter chosen to weight the penalty given to predictions compared to close versus far ground truth distances and $d(x_i, x_j)$ is the PHATE distance between cells i and j .

H.2 Negative Binomial Negative Log Likelihood Loss

Following Lopez et al. [25] and later Palma et al. [7], we use a negative binomial decoder for the autoencoder. The decoder produces a vector representing log-rate expression per-gene for each cell, which we convert to a per-gene mean expression parameter by applying an exponential per gene, $\mu_g = L \cdot e^{\text{log-rate}_g}$, where L is the input library size. This per-gene mean expression parameter is then used in combination with the per-gene (but not per-cell) dispersion to parameterize a negative binomial distribution for each gene in each cell. These negative binomial distributions can be sampled from to produce decoded cells with counts structure. The autoencoder is trained end-to-end by minimizing the sum of this NLL loss term and the distance preservation loss (described above). Gradients from the NLL loss term flow through both the decoder D_ψ and the encoder E_ϕ , while the distance loss acts only on the encoder (the gradients from distance loss skip the decoder). We explored an alternative parameterization of the decoder, which was subsequently discarded, where the decoder applied a softmax across genes to produce a probability simplex for each cell, before multiplying by L (see Chapter 6 for the motivation).

For a single gene g and input cell with discrete (unnormalized) counts x , we have the negative log likelihood for negative binomial distribution parameterized by $\mu_{x,g}$ and θ_g where the per-gene mean vector $\mu_x = D_\psi(E_\phi(x))$ with g -th entry $\mu_{x,g}$ and θ_g is learned per-gene:

$$\text{nll}(x^g, \mu_{x,g}, \theta_g) = -\log \Gamma(x^g + \theta_g) + \log \Gamma(\theta_g) + \log \Gamma(x^g + 1) - \theta_g \log \frac{\theta_g}{\theta_g + \mu_{x,g}} - x^g \log \frac{\mu_{x,g}}{\theta_g + \mu_{x,g}}$$

where x^g is the gene expression of gene g for cell x , and Γ is the gamma function, the continuous extension of the factorial satisfying $\Gamma(n) = (n-1)!$ for positive integers n .

Then, we average over all cells in the mini-batch during training to get the final negative binomial NLL loss:

$$\mathcal{L}_{\text{nll}}(\phi, \psi, \theta) = \frac{1}{B \cdot G} \sum_{i=1}^B \sum_{g=1}^G \text{nll}(x_i^g, \mu_{i,g}, \theta_g)$$

H.3 Vanilla (MSE) Autoencoder Training Objective

We train vanilla autoencoders with optional geometric regularization using the standard MSE loss function and compare results of generative flow models trained in the latent space of this autoencoder. The MSE loss function used is as follows:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{B \cdot G} \sum_{i=1}^B \sum_{g=1}^G (\hat{x}_i^g - x_i^g)^2$$

We use SoftPlus activation as a final step to ensure outputted predictions of log1p normalized expression values are non-negative.

I Condition Encoder Details

Let $\mathcal{B}$ be the condition batch containing p , the primary perturbation index, $\mathcal{K} = \{k_1, k_2, \dots, k_n\}$, a set of perturbation-specific covariate indices (covariates could be of different types, like drug, dose, etc. for a single cell), and $\mathcal{S} = \{s_1, s_2, \dots, s_m\}$, a set of sample-specific covariate indices (again, could be of different types, like replicate 1, cell type A, well 3). For the perturbation, $z_p = \sigma(W_p \cdot \text{Emb}(p) + b_p)$. For the perturbation covariates: $z_{k,i} = \sigma(W_{k,i} \cdot \text{Emb}(k_i) + b_{k,i})$ for $i \in \{1, \dots, n\}$. For the sample covariates: $z_{s,j} = \sigma(W_{s,j} \cdot \text{Emb}(s_j) + b_{s,j})$ for $j \in \{1, \dots, m\}$. The final output of the encoder is the result of passing the concatenation of the projections through the final output MLP (h_η).

$$C_\eta = h_\eta([z_p \oplus z_{k,1} \oplus \dots \oplus z_{k,n} \oplus \dots \oplus z_{s,m}])$$

The input to h_η is a fixed length, since each cell has one value for each type of covariate. We also ensure consistent ordering of covariates (i.e., perturbation always first in the input to h_η).

J Toy Dataset

J.1 Toy Dataset Methods

Each set of perturbed "cells" is one half of a noisy two-moons dataset from `sklearn.datasets`. The control cells are a Gaussian blob, with variance in the same order of magnitude as the width of the two-moons. We test FM model performance at different ambient dimensions, where we project the original 2D Gaussian to two-moons dataset using a linear orthogonal projection of dimension $D \times 2$. This allows us to evaluate the model performance in the original 2 dimensions using the inverse transformation. When training in PCA space, the FM model uses 2 principal components (2D space).

J.2 Toy Dataset Results

In Figure 1, we train a variety of Flow Matching (FM) models on a gaussian to two moons toy dataset, with two "perturbation" conditions, projecting the 2D data to different ambient dimensions. We find that flow matching model performance in the geometrically regularized latent space is better than in ambient space or in PCA space (2D), but worse than flow matching in the latent space of a vanilla autoencoder.

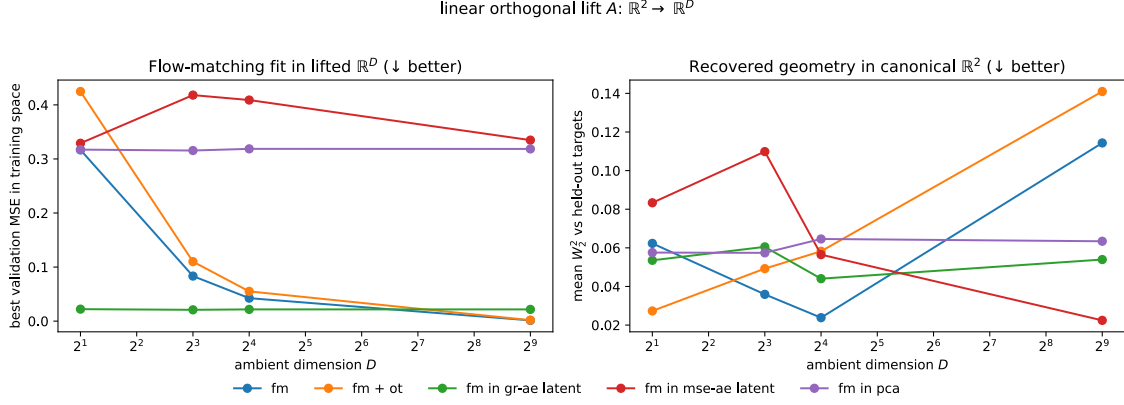


Figure 1: Plot of validation loss and W_2^2 in ambient space of different dimensionality for different flow matching models. The validation loss is the MSE loss used during flow matching model training, on the held-out samples. W_2^2 , shown on the right, is computed in the ambient space of varying dimensions. The `fm` model is a flow matching model with random source-target couplings, `fm + ot` adds optimal transport couplings and `fm in pca` trains in 2D PCA space. `fm in gr-ae latent` trains in the latent space of geometrically regularized autoencoder with latent dimension of 2 and `fm in mse-ae latent` trains in the latent space of a vanilla autoencoder with latent dimension of 2.

J.3 Toy Dataset Flow Matching Visualization

Figure 2 shows the matching plots to Figure 1, showing the predicted end distribution of perturbed ”cells” projected back into the original 2D space using the inverse of the linear orthogonal projection (see J.1). The top row shows the ground truth data, the columns show different ambient dimensions. The gray gaussian blob shown in each figure is the control (source) distribution, the green and blue moons shown in each figure are the two target distributions. The red dots are the predicted distribution supposed to match the green moon and the orange dots are the predicted distribution supposed to match the blue moon. It is clear from this figure, when comparing the Flow Matching (FM) in the geometrically regularized latent space (gr-ae latent; row 5), that this geometric regularization improves performance, when compared to flow matching in the ambient space (just fm; row 2). Yet, the flow matching model in MSE autoencoder latent space is very good, achieving the best W_2^2 in 512-D space. We show in 5.2 that an autoencoder trained with MSE loss is optimal for W_2^2 (comparing predicted reconstruction of all data to ground truth), which fits with this result on the toy dataset.

K Dimensionality Reduction as a Bottleneck on Model Performance Contd.

In Appendix A we derive theoretical results for the expected Cosine LogFC and W_2 metrics as a function of total variance TV and cumulative variance explained CVE_k which closely match the theoretical results in 3 discussed above. Note that the data is assumed to be Gaussian in the proof for the W_2 distance and there are assumptions in both proofs

that rely on the dimensionality reduction being PCA, so this is not a general statement about cumulative variance explained.

For Cosine LogFC, $\mathbb{E}[\text{CosineLogFC}] \approx \sqrt{\text{CVE}_k}$, expanding the formula for Cosine LogFC and using the idempotency of P_k ($P_k^2 = P_k$) followed by linearity of expectation and trace simplifications. This result states that the expectation of Cosine LogFC over all perturbation covariate combinations (drug and dose) is approximately the square root of the cumulative variance explained by PCA using the top k components (CVE_k). CVE_k is the ratio of explained variance by the top k principal components relative to the total variance of the data. This is almost exactly the relationship seen experimentally shown in 3, where Cosine LogFC scales with the square root of variance explained (power law with exponent of 0.5). The discrepancy between theoretical and experimental exponents could be due to the non-linear $\log_{1p}$ transformation applied to the counts before PCA and the non-Gaussian structure of single-cell data.

This theoretical result shows a latent-space reconstruction ceiling on performance of any model performing perturbation prediction in PCA space on the Cosine LogFC metric, suggesting that this latent-space reconstruction ceiling must be carefully considered when comparing model performance and optimizing for Cosine LogFC. A model bounded by CVE_k , like when predicting in PCA space, is biased towards predicting global transcriptional shifts and is bottlenecked from predicting subtle perturbations. One example of a perturbation prediction model that predicts in PCA space is CellFlow [17], which performs Flow Matching in either PCA space or the latent space of a VAE. The authors found that PCA was more powerful than VAEs for distributional metrics.

For W_2 , we find that the latent-space reconstruction limit of perturbation prediction performance is $W_2 = \sqrt{\text{TV}} \cdot \sqrt{1 - \text{CVE}_k}$, which is shown in Appendix A, where TV is total variance in the data and CVE_k is cumulative variance explained by the top k PCs, as above. Since TV does not depend on CVE_k or k itself (total variance is intrinsic to the data), this can be interpreted as saying that the latent-space reconstruction floor of W_2 in PCA space is proportional to the square root of $1 - \text{CVE}_k$, a power law relationship wrt $1 - \text{CVE}_k$. This is also close to what is seen experimentally, where the latent-space reconstruction floor of W_2 decreases (W_2 is a distance so lower is better) as the cumulative variance explained increases (see Figure 3). Note that this assumes the underlying data is Gaussian to use the Bures/Fréchet distance.

Directly from the definition of W_2^2 distance we have:

$$W_2^2(\rho, \nu) = \inf_{(X, X') \sim \gamma \in \Pi(\rho, \nu)} \mathbb{E}_\gamma[\|X - X'\|^2] \leq \mathbb{E}[\|X - g(f(X))\|^2]$$

where $X' = g(f(X))$. This is stating that the smallest cost coupling between X and X' is no worse than the specific coupling $X \mapsto g(f(X))$. If we look at $\mathbb{E}[\|X - g(f(X))\|^2]$, this is simply MSE reconstruction loss of an autoencoder. This means that, while our autoencoder trained with only MSE loss (as is standard) on the Sciplex3 dataset is not optimized only to minimize W_2^2 between the ground truth distribution of perturbed cells and the predicted distribution of perturbed cells (as there are other cells the model trains on), one would expect an autoencoder trained with MSE to be particularly well suited for the W_2^2 metric. This is in fact exactly what we see in 5.1, where the flow matching model trained in the latent space of the vanilla autoencoder trained with MSE loss has better performance on the W_2^2 metric compared to generative flow models trained in other latent spaces.

Klein et al. [17] also find that the dimensionality and space of prediction bottlenecks the latent-space reconstruction limit of model performance on different perturbation prediction metrics. They show experimentally that the upper bound computed by encoding and decoding the true perturbed cells improves as the number of PCs kept increases, across both distributional metrics like Energy Distance, MMD and Sinkhorn divergence, as well as mean-expression metrics like R squared and R squared DEG. Interestingly, they find that across all distributional metrics tested, the latent-space reconstruction limit of training in latent space of a VAE is worse than just 50 PCs. They also show that their CellFlow model achieves significantly worse performance across all metrics than the latent-space reconstruction limit of the VAE or 50 PCs models, suggesting a significant gap between current perturbation prediction methods and limits to model performance.

L Additional Figures

M Additional Tables

Table 2: Comparison of related continuous flow/geometry-aware/ perturbation-prediction models.

Model	Continuous Flow?	Geometry Aware?	Perturbation Prediction?
CPA	No	No	Yes
CellFlow	Yes	No	Yes
FlatVI	Yes	Yes	No
MIOFlow	Yes	Yes	No
GAGA	Yes	Yes	No
CFlows	Yes	Yes	No
GLFM (Ours)	Yes	Yes	Yes

M.1 Comparison with PerturBench Results

In Table 3, we adapt our models to train with the PerturBench codebase, still with the Sciplex3 dataset. All metrics are from PerturBench [12] and are not re-implemented in our code. We follow the covariate-transfer protocol on the Srivatsan20 (SciPlex3) dataset (single fold, 30% perturbations held out for each cell type, top dose only); see below for more details. We don’t show GLFM trained in the geometrically regularized latent space, as we found that it underperformed models trained in 16-d PCA space.

Note that GLFM models (FM, Neural ODE only) are trained in 16-dimensional PCA space when denoted with "(PCA 16)" for this comparison, and in full HVG+DEG gene space otherwise. We have rerun PerturBench baselines and find our rerun results closely match with the reported results in the original paper [12].

For the direct evaluation on PerturBench, we use their code and metrics. The PerturBench setup only uses the top dose for each perturbation for the SciPlex3 dataset. Each perturbation is held out in at most one cell type, but the single split evaluated has

Table 3: **Covariate transfer results on Srivatsan20 (sci-Plex3) from PerturBench eval:** Mean $\pm$ STD reported over held-out perturbations, with variance based on 3 seeds, not cross validation. **Bold** is best model, underlined is second best, per metric. Perturbations are held out for each cell type (perturbation is held out in at most one cell type). Perturbations are of highest dose only.

Model	Cosine LogFC ($\uparrow$)	MMD PCA ($\downarrow$)	Top- k Recall ($\uparrow$)
<i>PerturBench baselines</i>			
CPA	0.3536 ± 0.0056	0.6039 ± 0.0235	0.0002 ± 0.0001
DecoderOnly	0.3333 ± 0.0136	1.8697 ± 0.0053	0.0000 ± 0.0000
LatentAdditive	0.4853 ± 0.0003	1.8080 ± 0.0028	0.0000 ± 0.0000
<i>GLFM</i>			
Decoder	0.3898 ± 0.0009	1.5760 ± 0.0073	0.0000 ± 0.0000
FM	0.0712 ± 0.0018	0.2781 ± 0.0098	0.0079 ± 0.0005
FM (PCA 16)	0.3569 ± 0.0124	0.5041 ± 0.0136	0.0000 ± 0.0000
Neural ODE	0.1291 ± 0.0065	<u>0.3099 ± 0.0010</u>	<u>0.0030 ± 0.0013</u>
Neural ODE (PCA 16)	<u>0.3914 ± 0.0052</u>	0.5579 ± 0.0023	0.0000 ± 0.0000

held out perturbations on all cell types, unlike our standard split which only holds out perturbations on K562. Up to 30% of perturbations are held out in each cell type.

Table 4: Flow matching in 16-d PCA space. Standard models have 4 hidden layers of dimension 256, the tiny model has 2 hidden layers of dimension 64. Mean $\pm$ STD reported over 375 held-out perturbations in K562 cell line of Sciplex3 dataset, variance based on seed, not cross validation. **Bold** is best model, underlined is second best, per metric.

Variant	Mean Gene $\bar{W}_1$ ($\downarrow$)	W_2^2 ($\downarrow$)	Cosine LogFC ($\uparrow$)
FM (no OT, standard)	0.2482 ± 0.0074	2134.9 ± 64.6	0.1585 ± 0.1149
FM (OT, standard)	0.2487 ± 0.0078	2136.3 ± 64.6	0.1689 ± 0.1070
FM (OT, tiny)	0.2456 ± 0.0071	2140.0 ± 63.2	0.1771 ± 0.1039

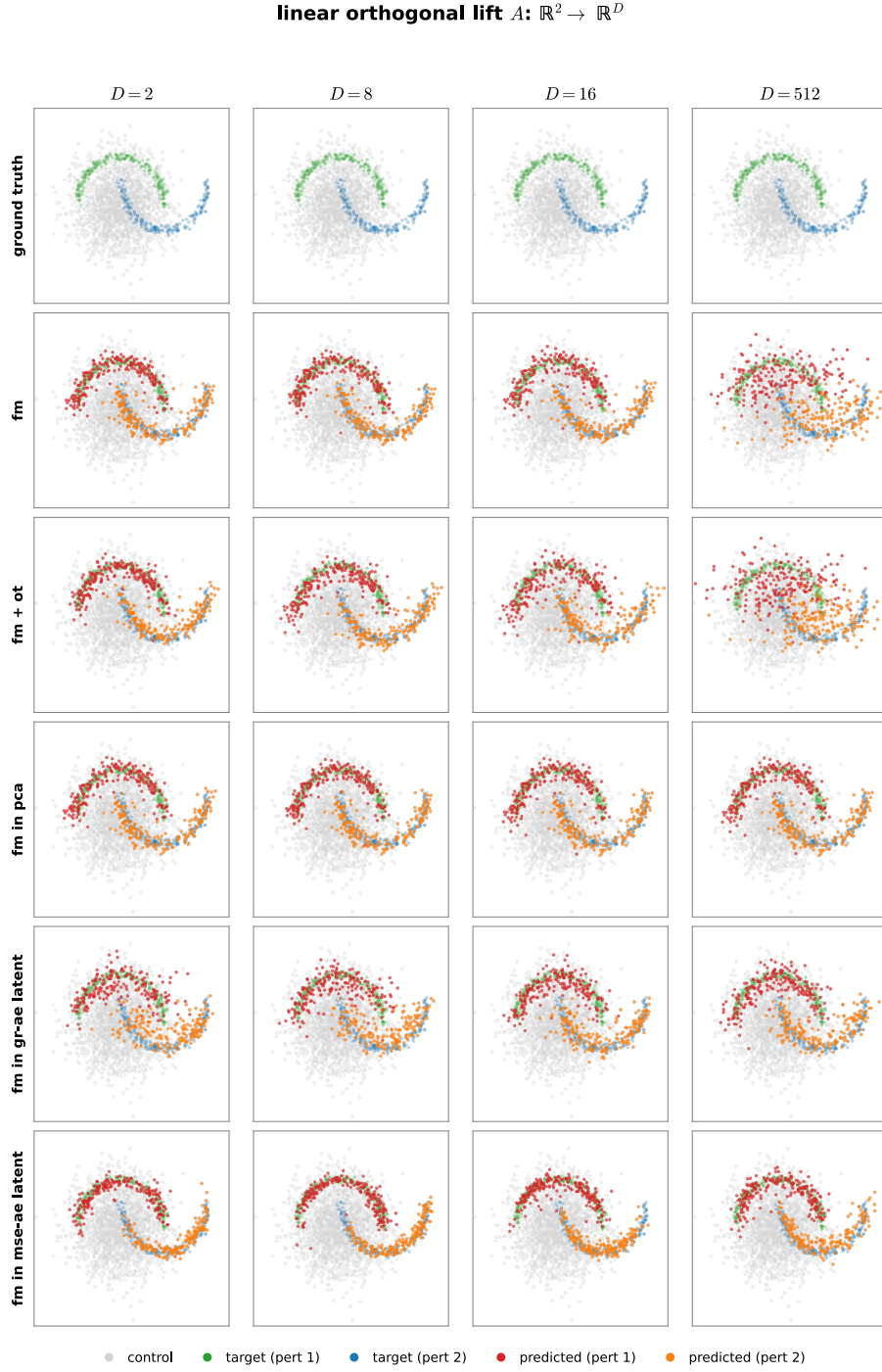


Figure 2: Predicted vs ground truth for different flow models on toy dataset. Predictions are projected back to the original 2D data coordinates for visualization using the inverse linear orthogonal projection.

theoretical reconstruction limits metric vs variance explained

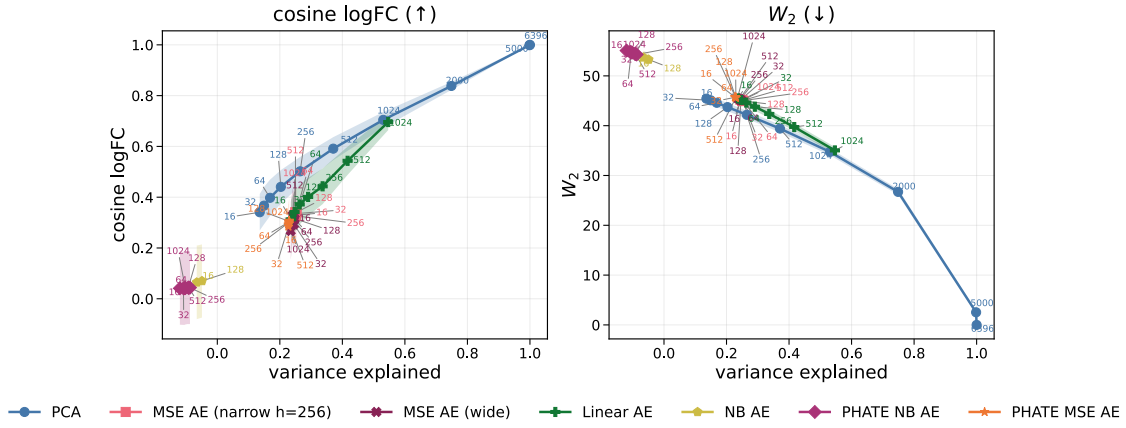


Figure 3: Latent-space reconstruction limit of model performance at different PCA dimensions on Cosine LogFC and W_2^2 metrics, compared to variance explained. Variance explained is on the x axis, the labels above the points represent the dimension of PCA used. Shaded area represents ± 1 STD calculated over the perturbation covariate combinations (drug and dose). For this comparison, both metrics are computed in gene space (after normalization and HVG+DEG selection) rather than W_2 being computed in PCA space as is done for the evaluation metric W_2^2 on perturbation models. Flow matching models trained without optimal transport couplings.

FM + ODE performance on disjoint3 split pooled across cell types

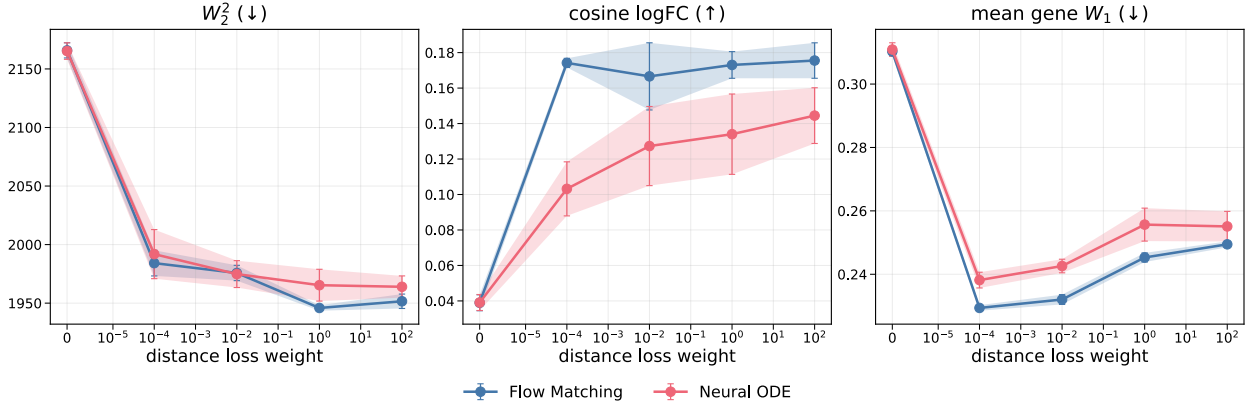


Figure 4: Performance of flow matching and Neural ODE models trained in latent space of a vanilla (MSE loss) autoencoder as a function of geometric regularization strength. The shaded region represents the $\pm$ std across seeds, where each seed determines train-val-test split of perturbations. Unlike the standard split used in this paper, this figure represents a setting where 30% of perturbations are held-out for each cell type in a single seed, where a perturbation is held-out for at most one cell type, so it is present in the other two cell types in the training data if it is held-out. This figure is meant to demonstrate that the results shown in Figure 5.3 are not specific to holding out the K562 cell-type.

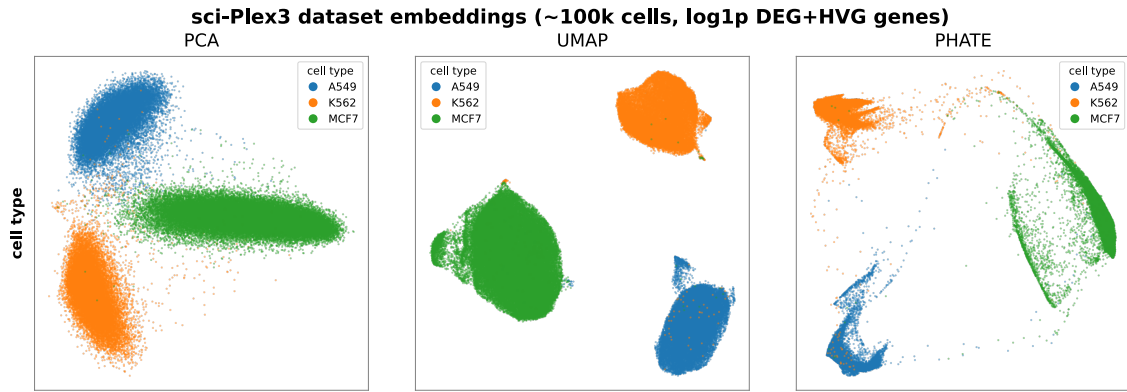


Figure 5: Embeddings of SciPlex3 Dataset Colored by Cell Type: PCA, UMAP, and PHATE embeddings of the SciPlex3 dataset, 100k cell subset. PHATE is performed on 50 PCs.

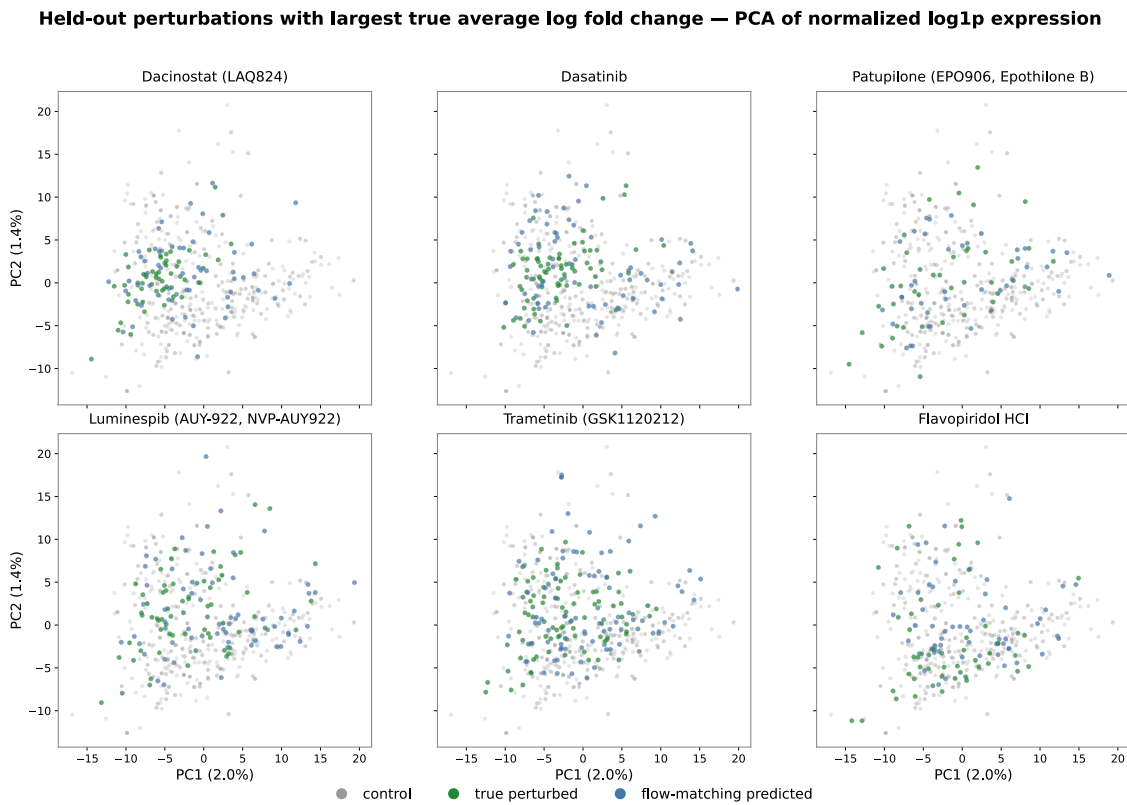


Figure 6: PC1 vs PC2 for control cells, ground truth perturbed cells, and predicted perturbed cells for top-effect K562 held-out perturbations.